

Introduction to Jython

What is Jython?

Jython is an implementation of the Python programming language that runs on the **Java platform**. It allows you to write Python code that can directly interact with Java classes and libraries.

👉 In simple terms:

Jython = Python + Java Integration

Basics of Jython

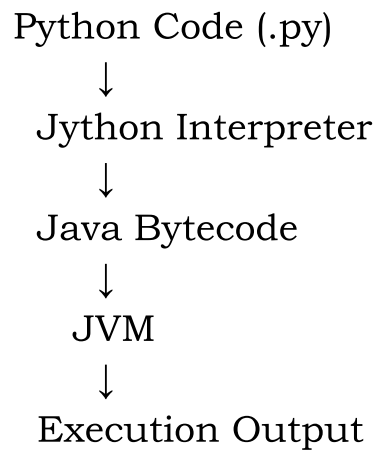
1. Key Features

- ✓ Written in Java
 - ✓ Runs on the **Java Virtual Machine (JVM)**
 - ✓ Can import and use Java classes directly
 - ✓ Combines Python simplicity with Java power
 - ✓ Platform-independent (like Java)
-

2. How Jython Works

- Python code is interpreted and executed on JVM
 - It converts Python code into Java bytecode internally
 - Allows seamless communication between Python and Java
-

3. Jython Architecture



4. Installing Jython

Steps:

1. Download Jython installer (JAR file)
2. Run using:

```
java -jar jython-installer.jar
```

3. Set environment variables (optional)

5. Basic Syntax Example

Simple Python Code in Jython

```
print("Hello from Jython!")
```

6. Using Java in Jython

Example: Import Java Class

```
from java.lang import System
```

```
System.out.println("Hello from Java inside Jython")
```

👉 This is the main advantage of Jython — using Java libraries in Python code.

7. Working with Java Objects

```
from java.util import Date
```

```
d = Date()
print(d)
```

8. Differences: Jython vs Python

Feature	Python (CPython)	Jython
Platform	Native	JVM
Java Support	Limited	Full

Speed	Fast	Slightly slower
Libraries	Python libs	Java + Python

9. Advantages of Jython

- Easy integration with Java projects
 - Reuse Java libraries
 - Great for enterprise applications
 - Simplifies scripting in Java environments
-

10. Limitations

- ❌ Does not support CPython C extensions
 - ❌ Slightly slower than native Python
 - ❌ Limited support for latest Python versions
-

Summary

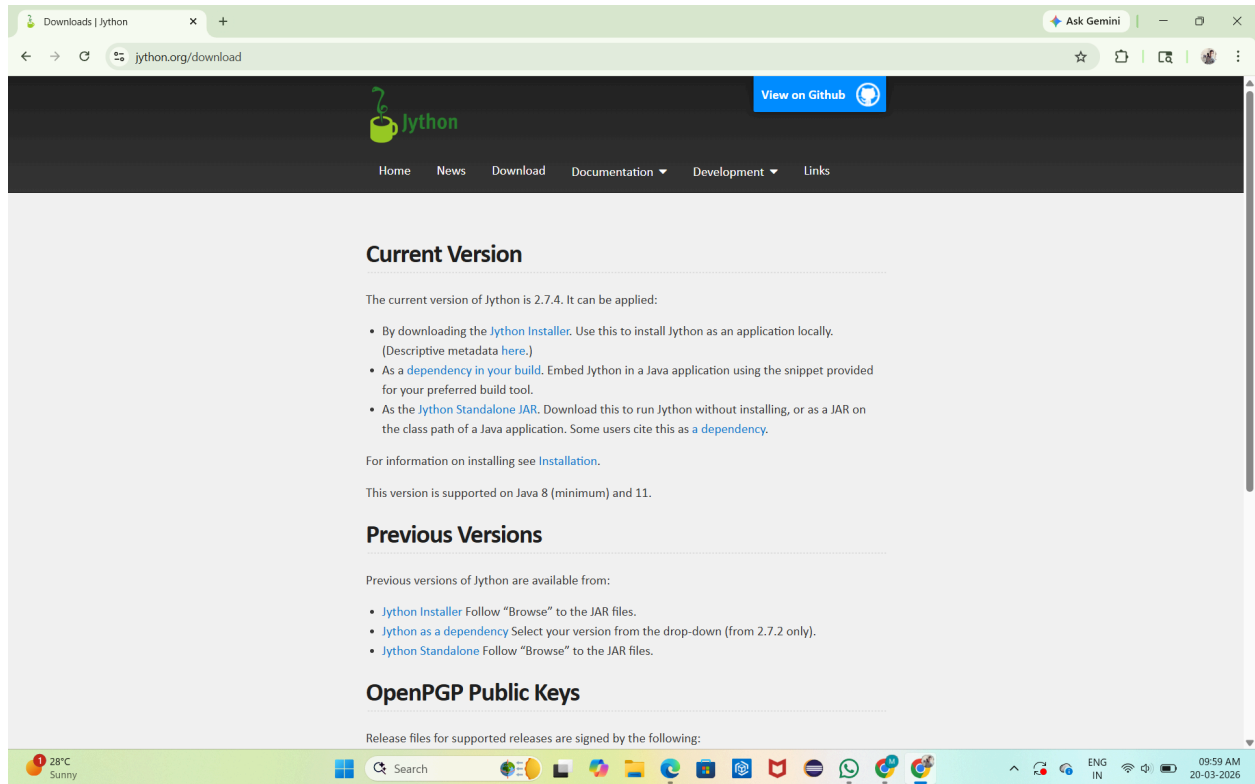
Jython is a powerful tool that bridges Python and Java, allowing developers to write Python code while leveraging Java's extensive ecosystem. It is especially useful in enterprise applications where Java is dominant.

Jython Installation & Setup Guide

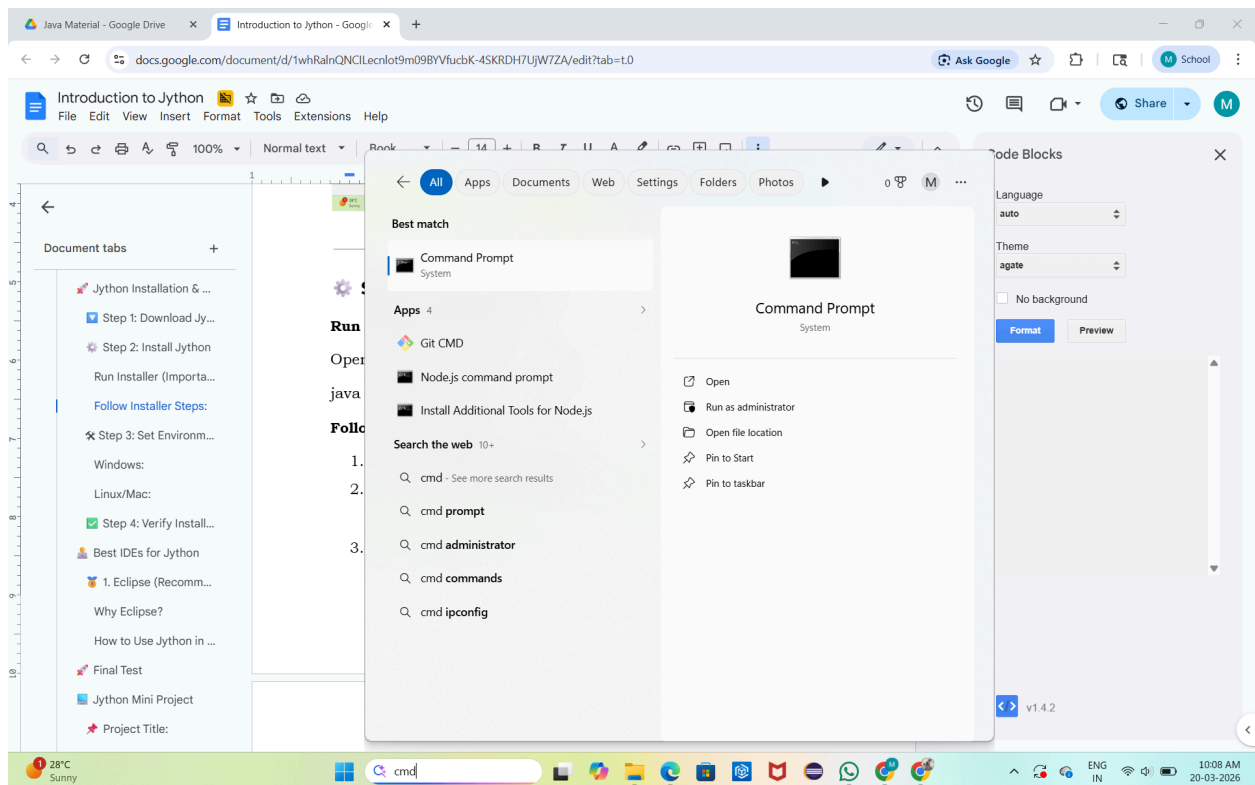
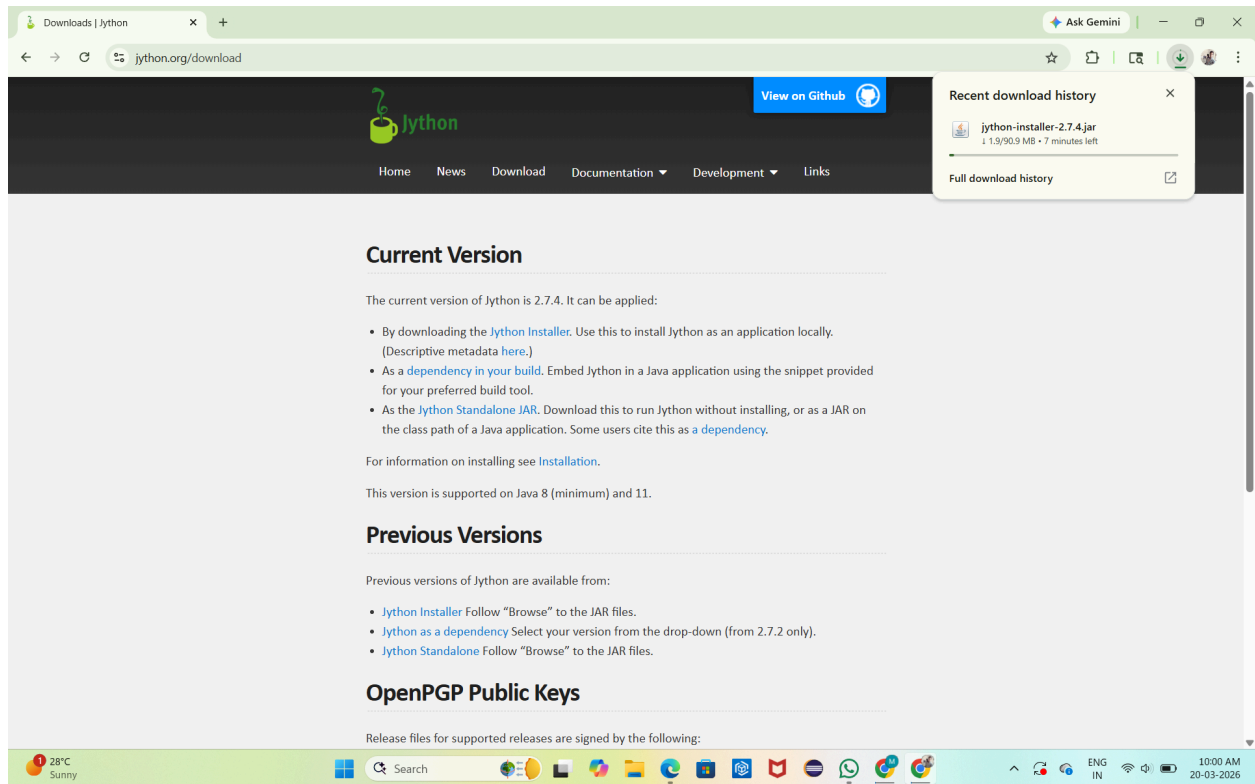
Step 1: Download Jython

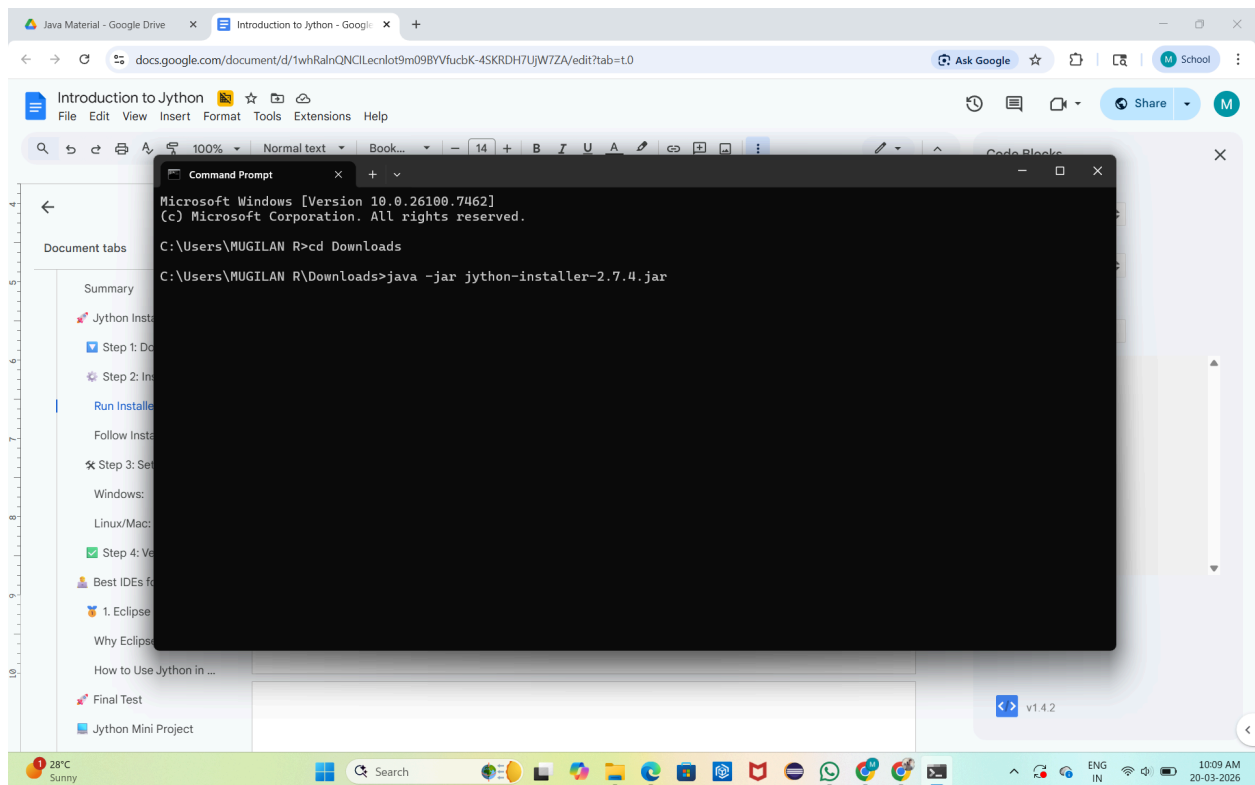
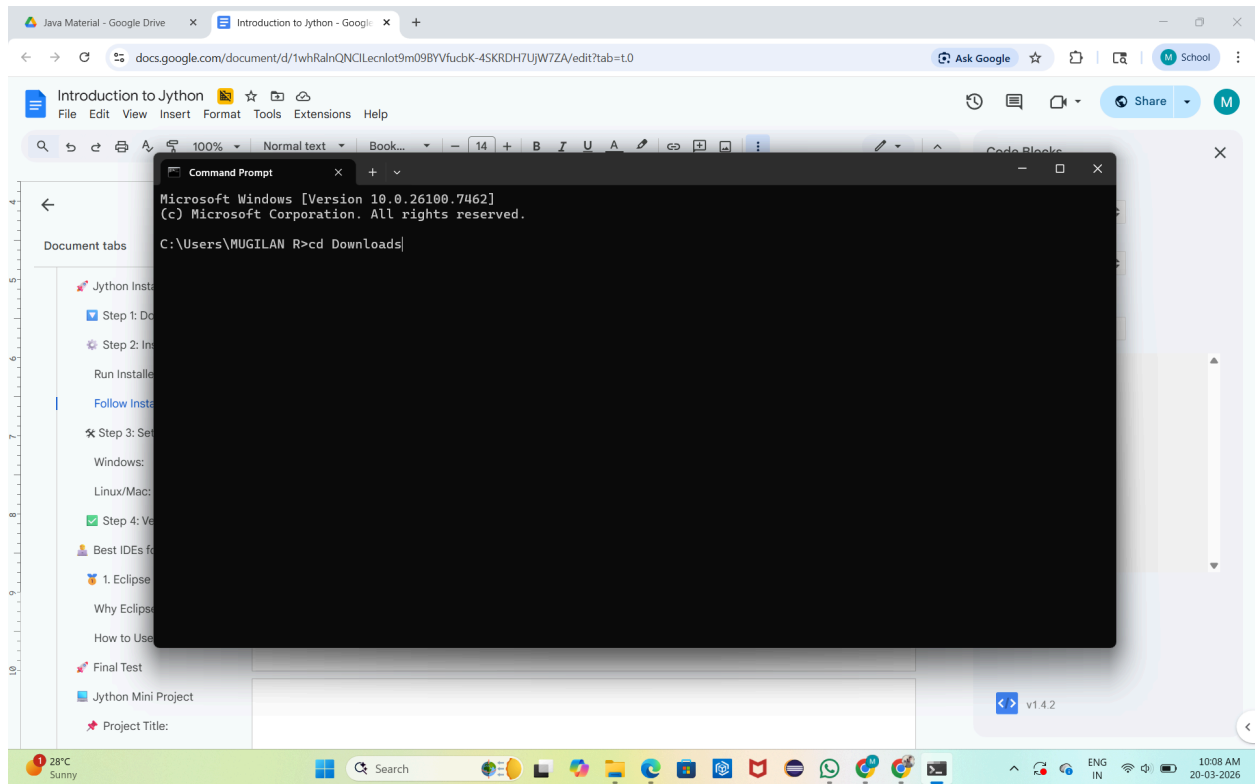
- Go to the official Jython website:

 <https://www.jython.org/download>



- Click on the Jython Installer in the Current Version
- Download:
`jython-installer-2.7.x.jar`



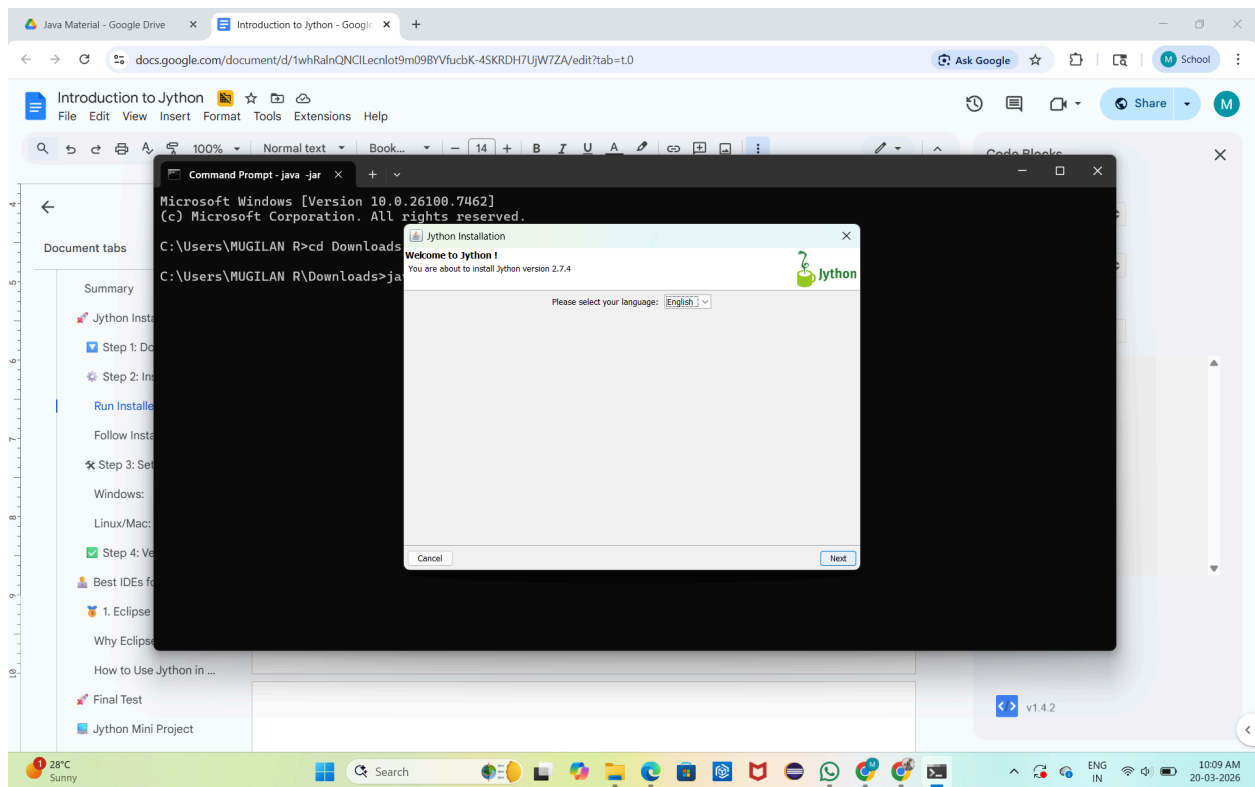


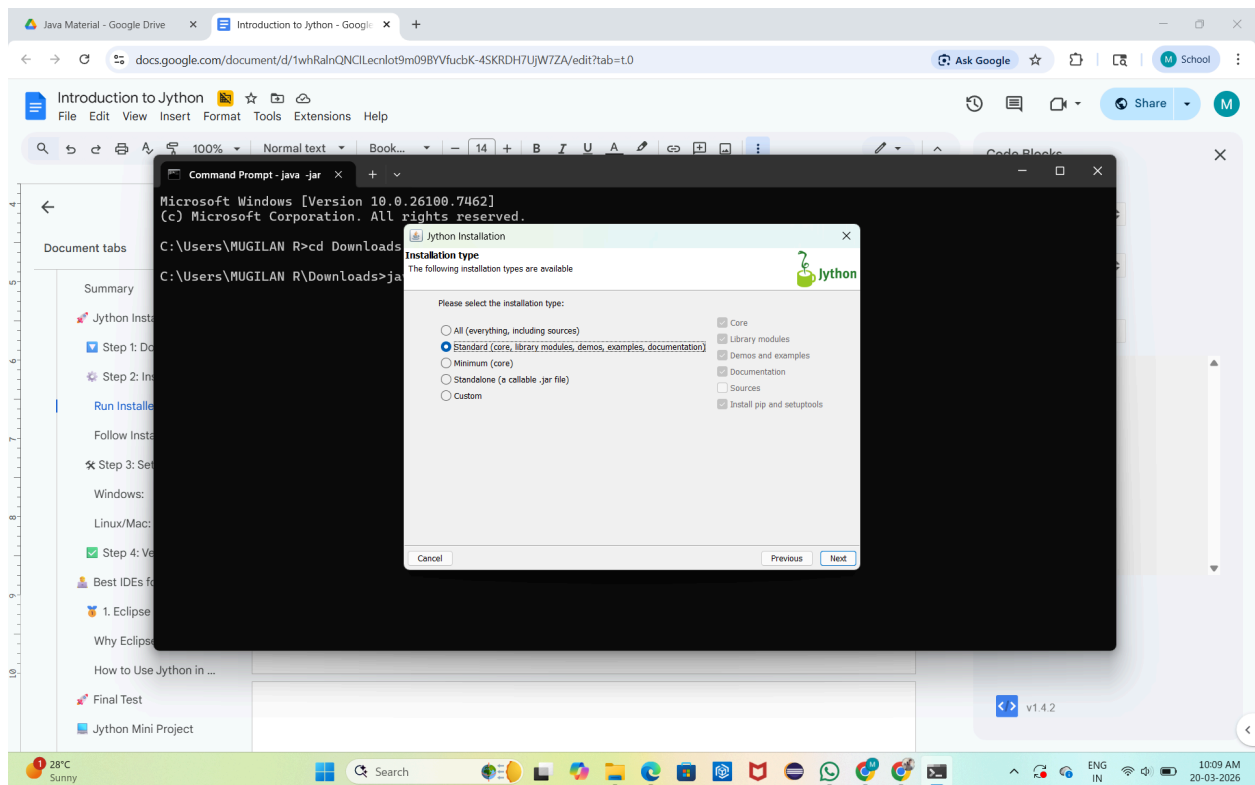
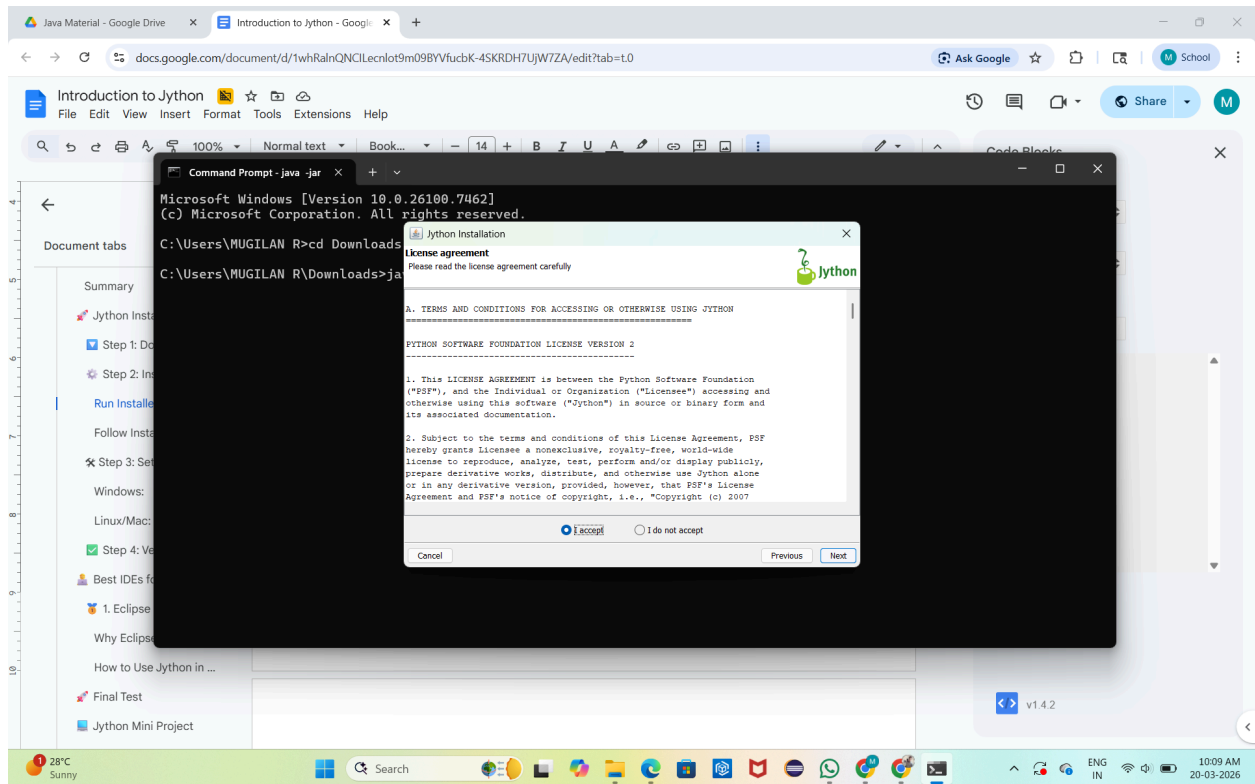
⚙️ Step 2: Install Jython

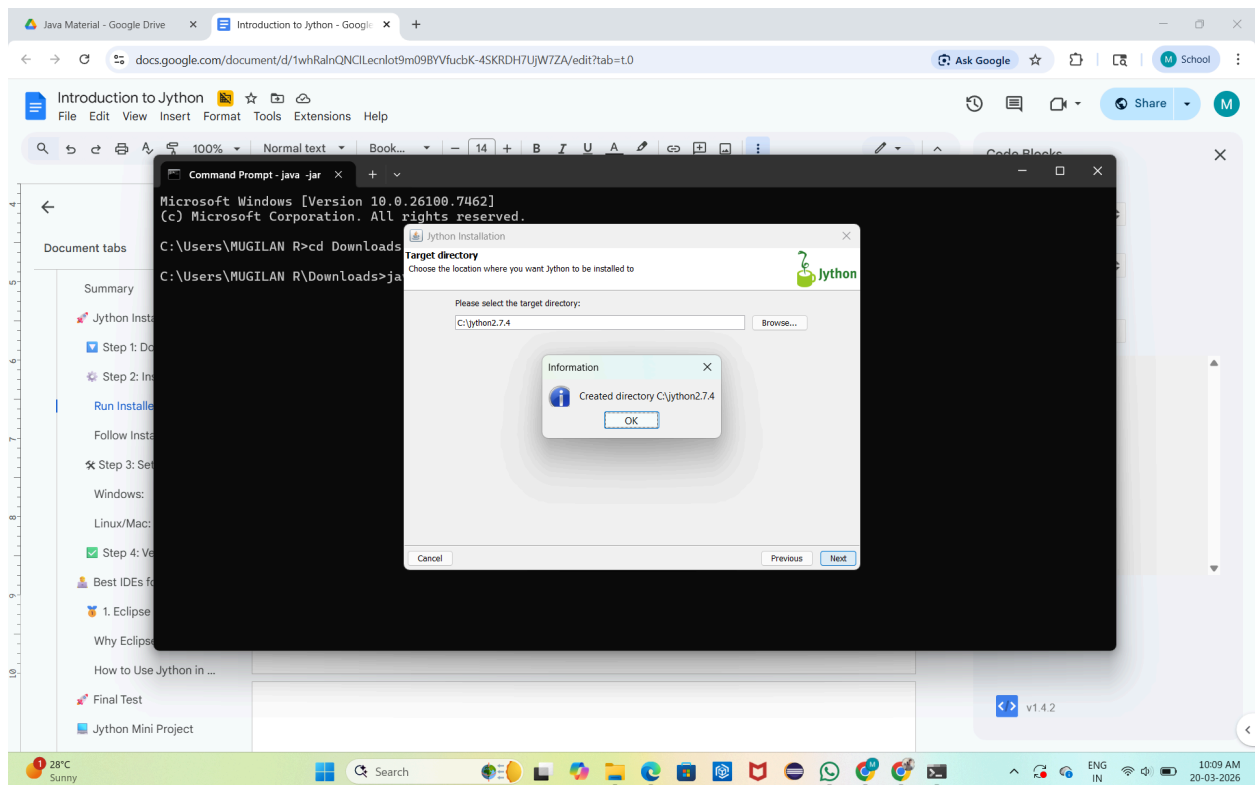
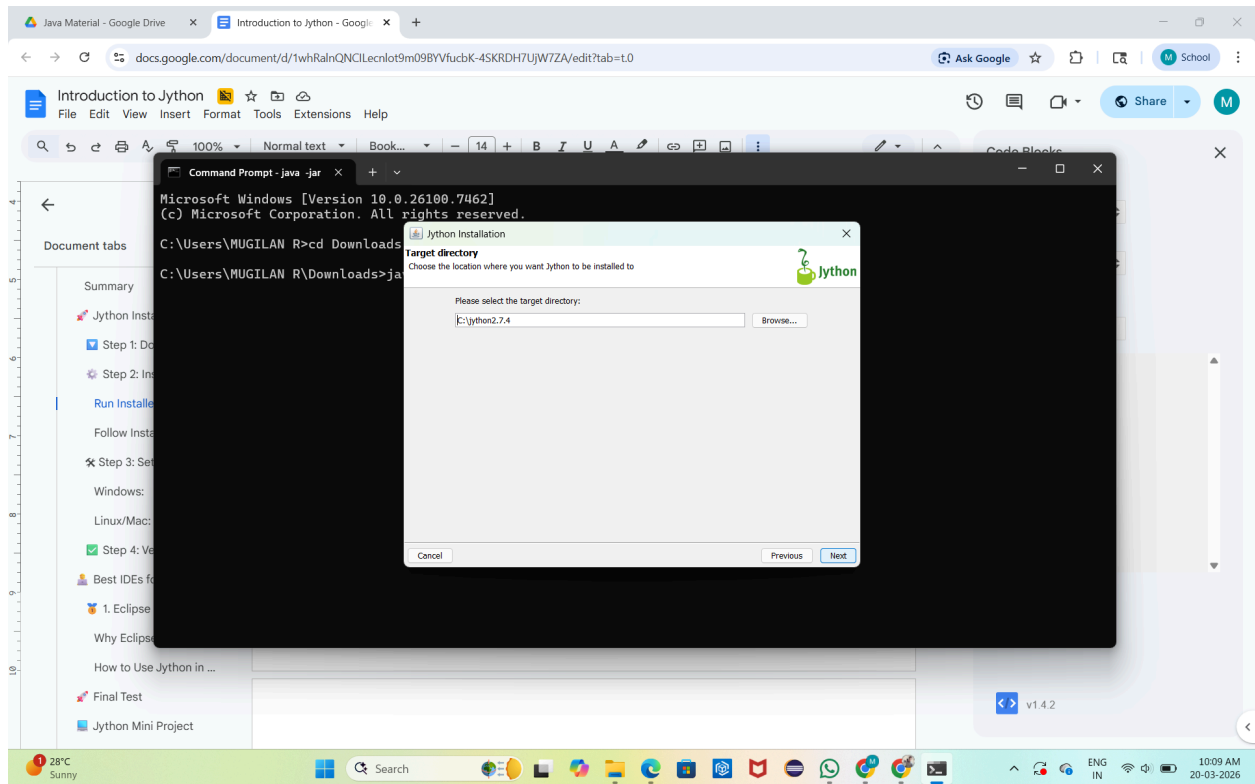
Run Installer (Important)

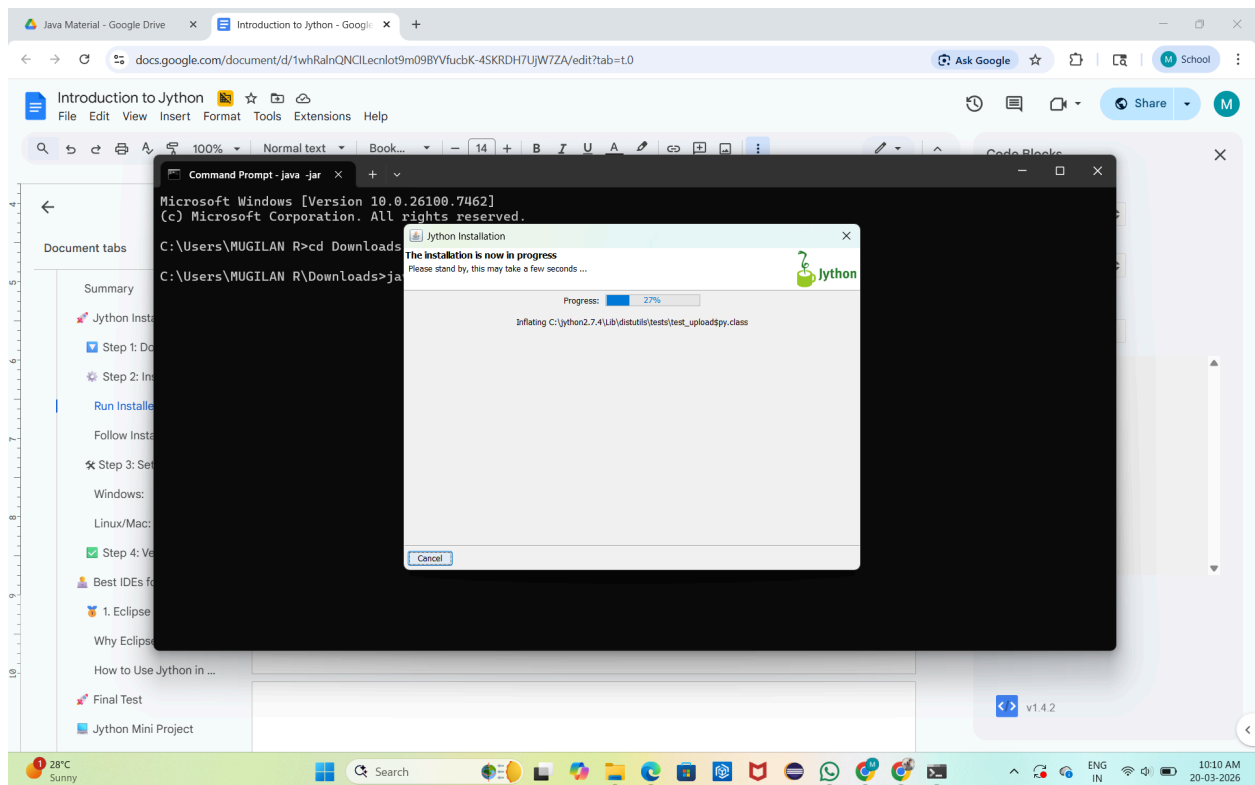
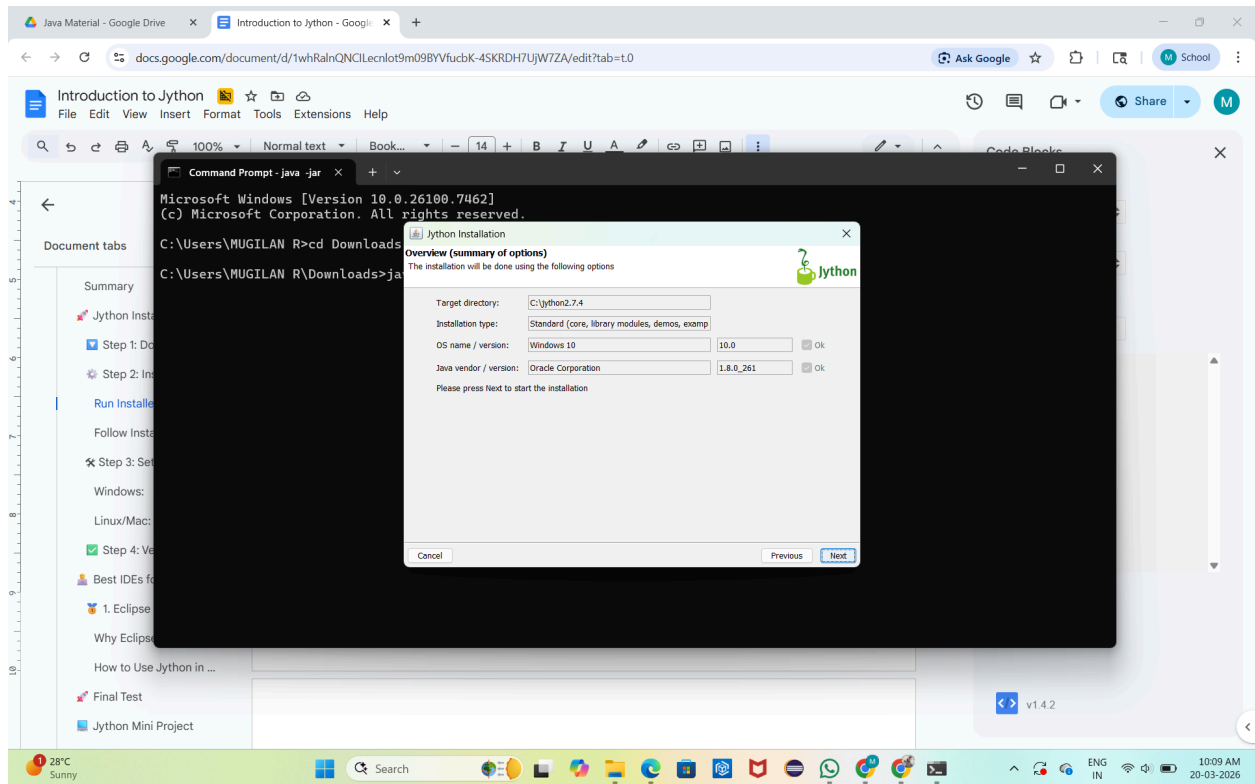
Open terminal / command prompt:

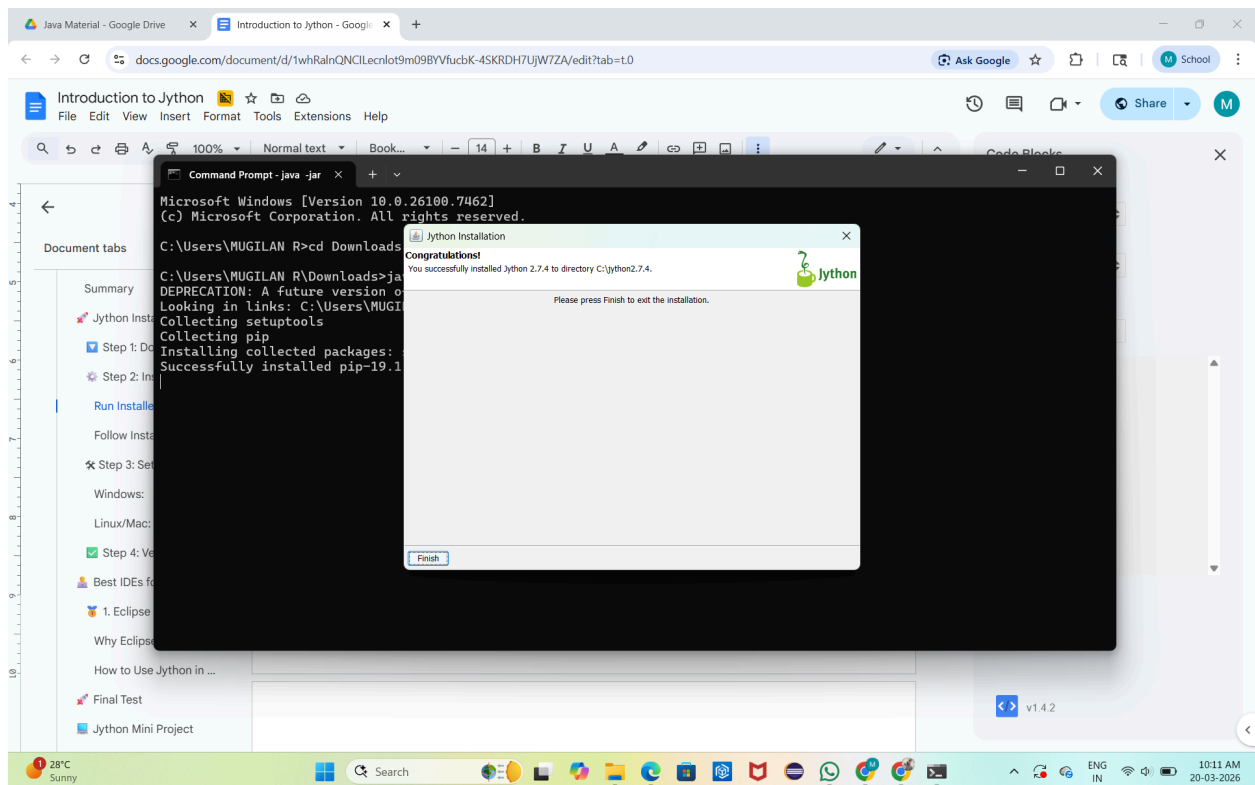
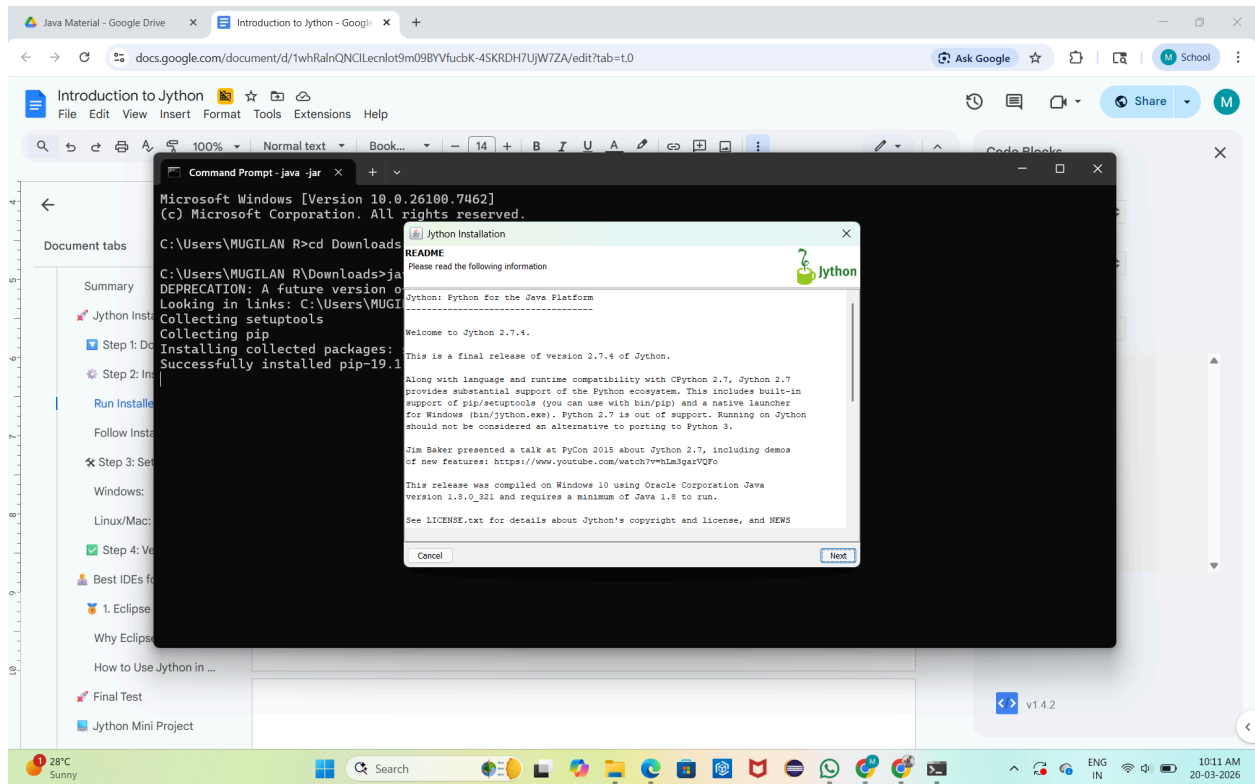
```
java -jar jython-installer-2.7.4.jar
```

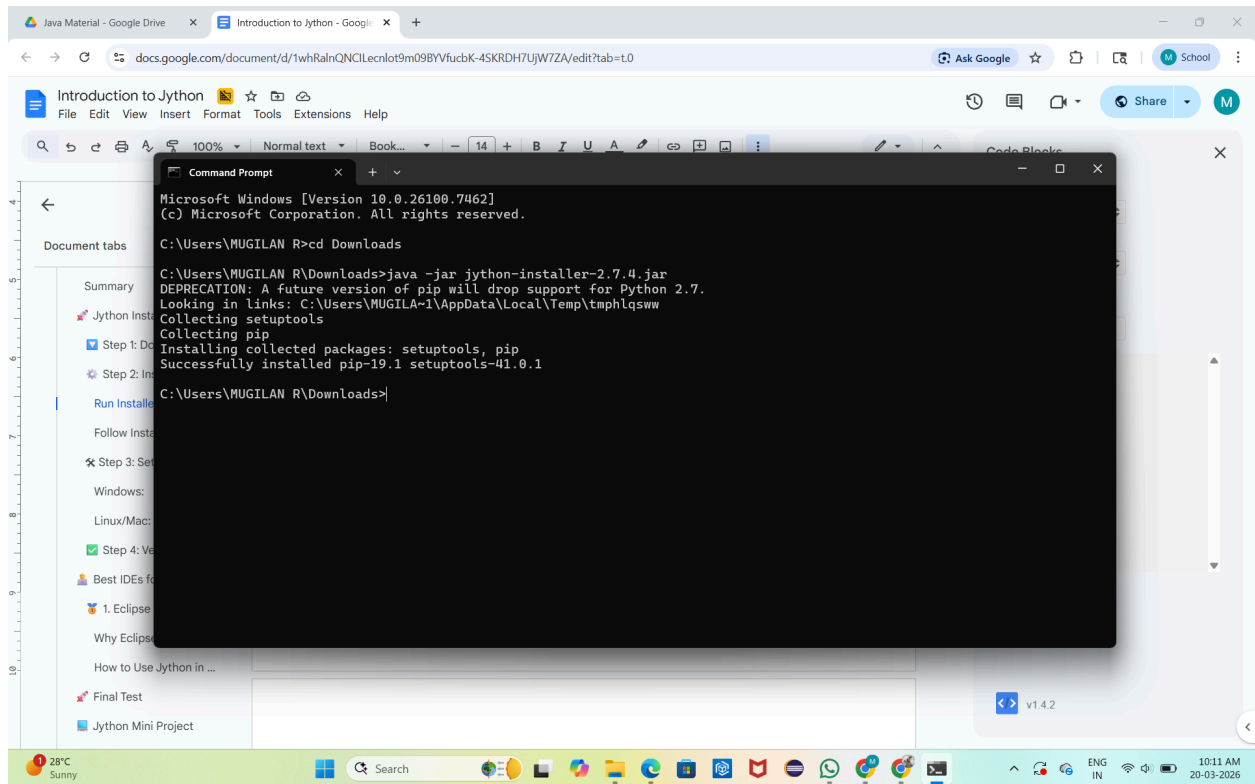












Follow Installer Steps:

1. Choose **Standard Installation**
2. Select installation directory (e.g., `C:\Jython` or `/usr/local/jython`)
3. Finish installation

Step 3: Set Environment Variables

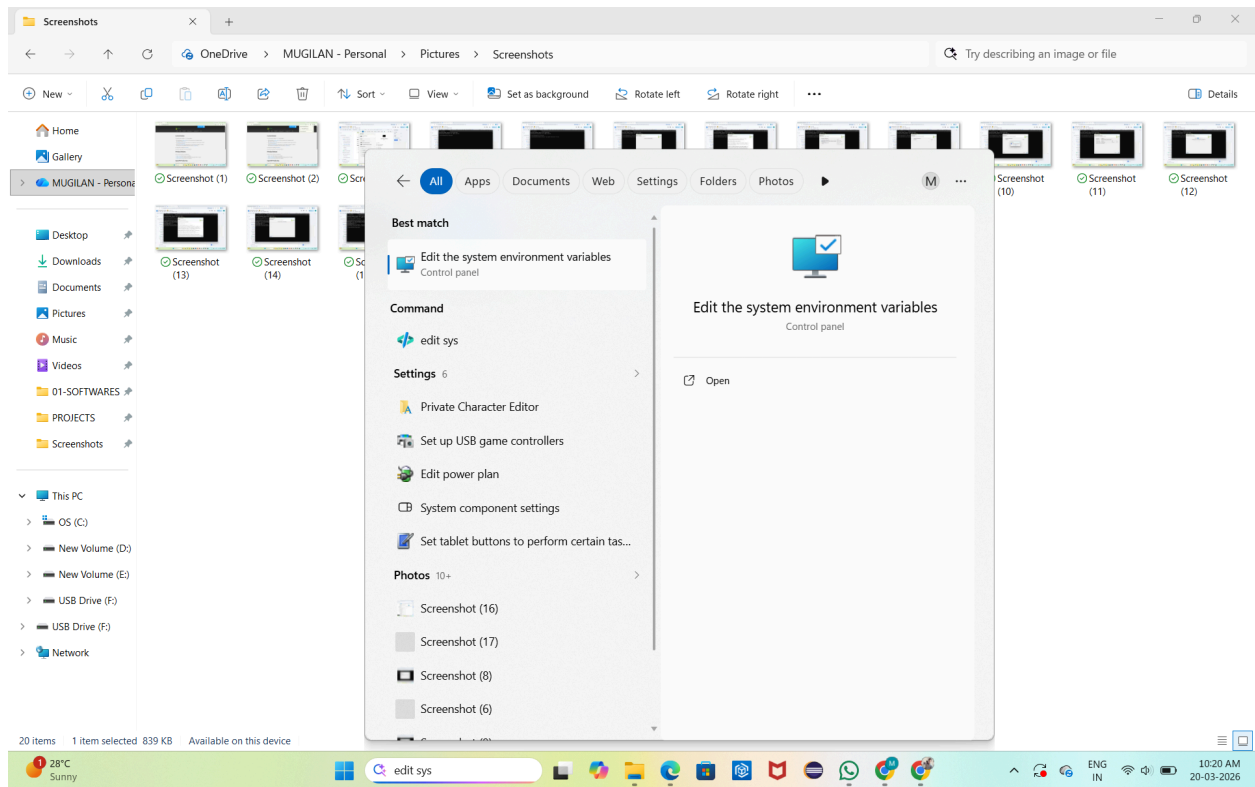
Windows:

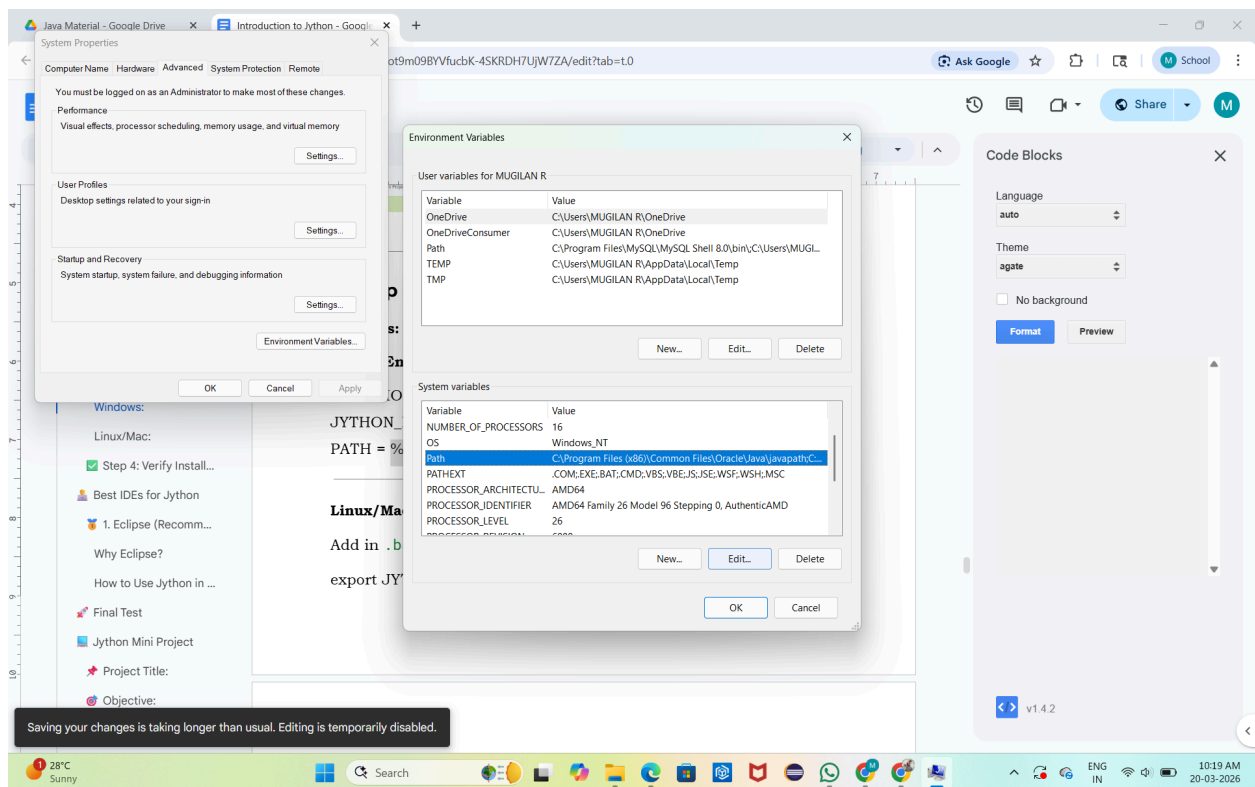
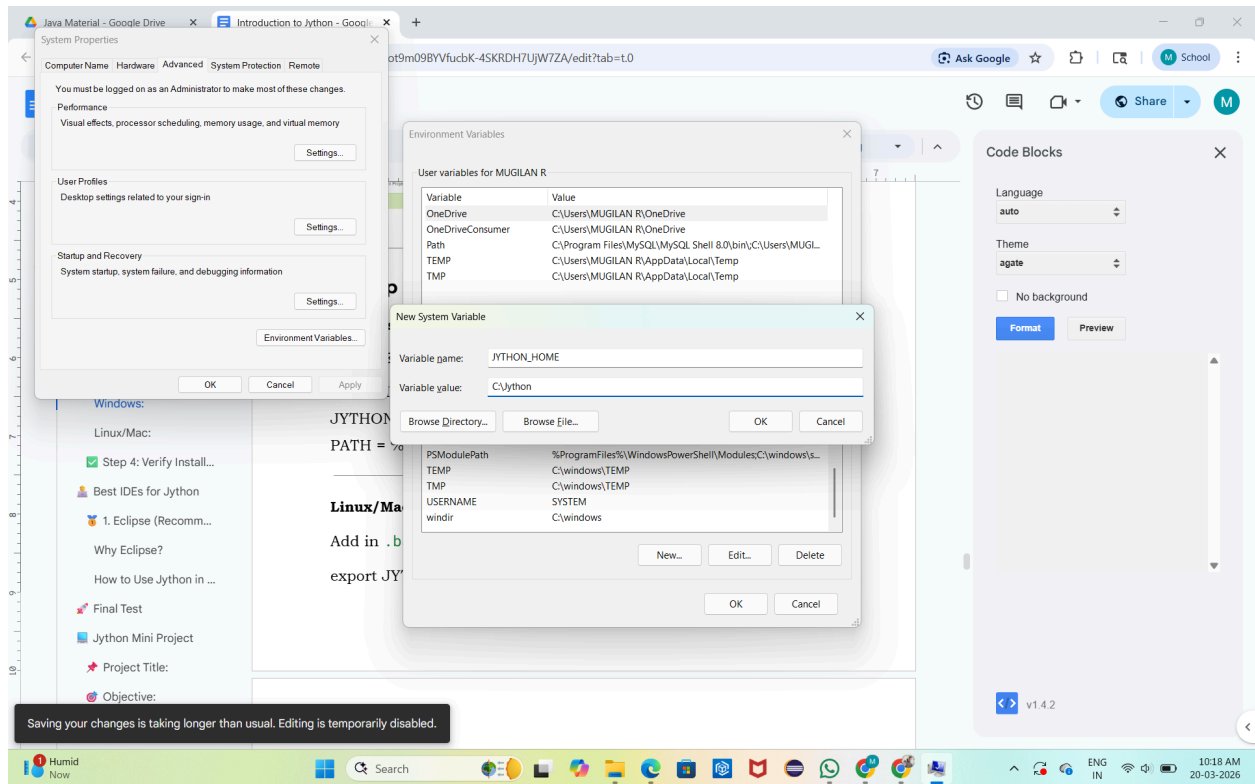
Add to **Environment Variables**:

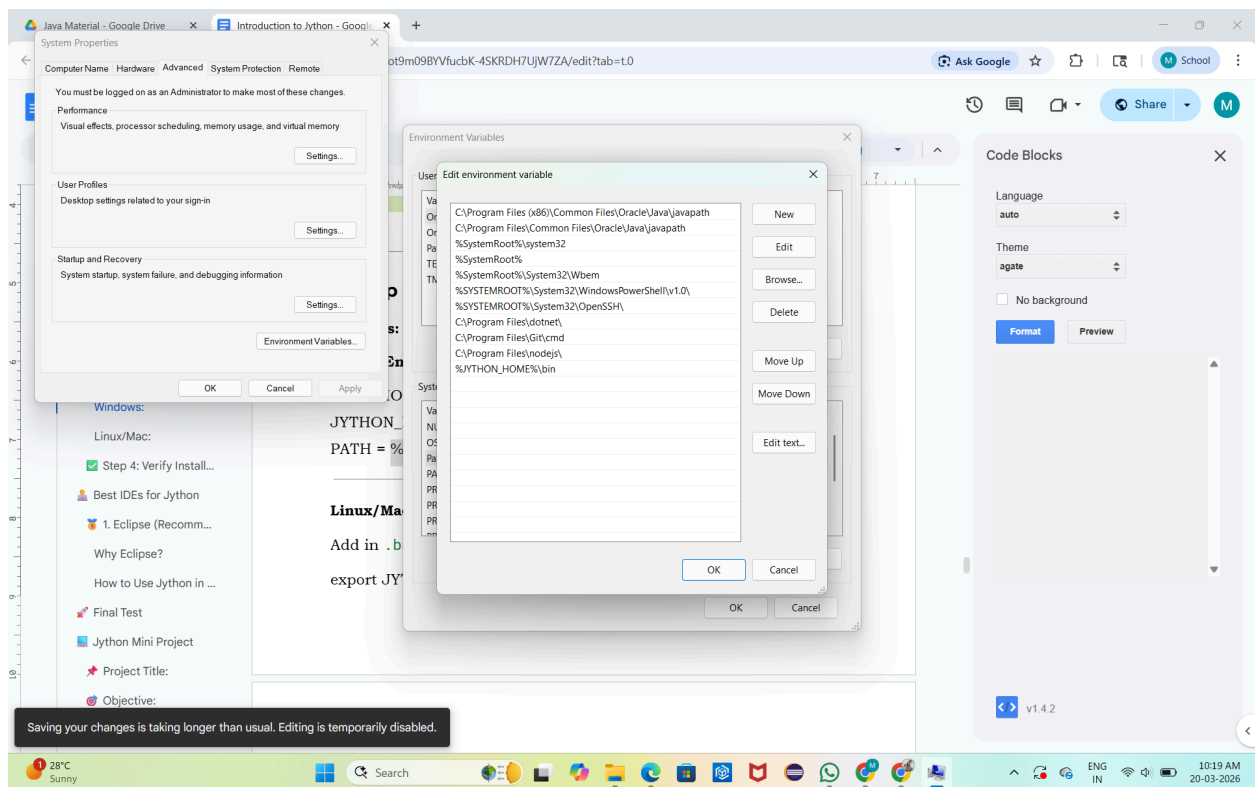
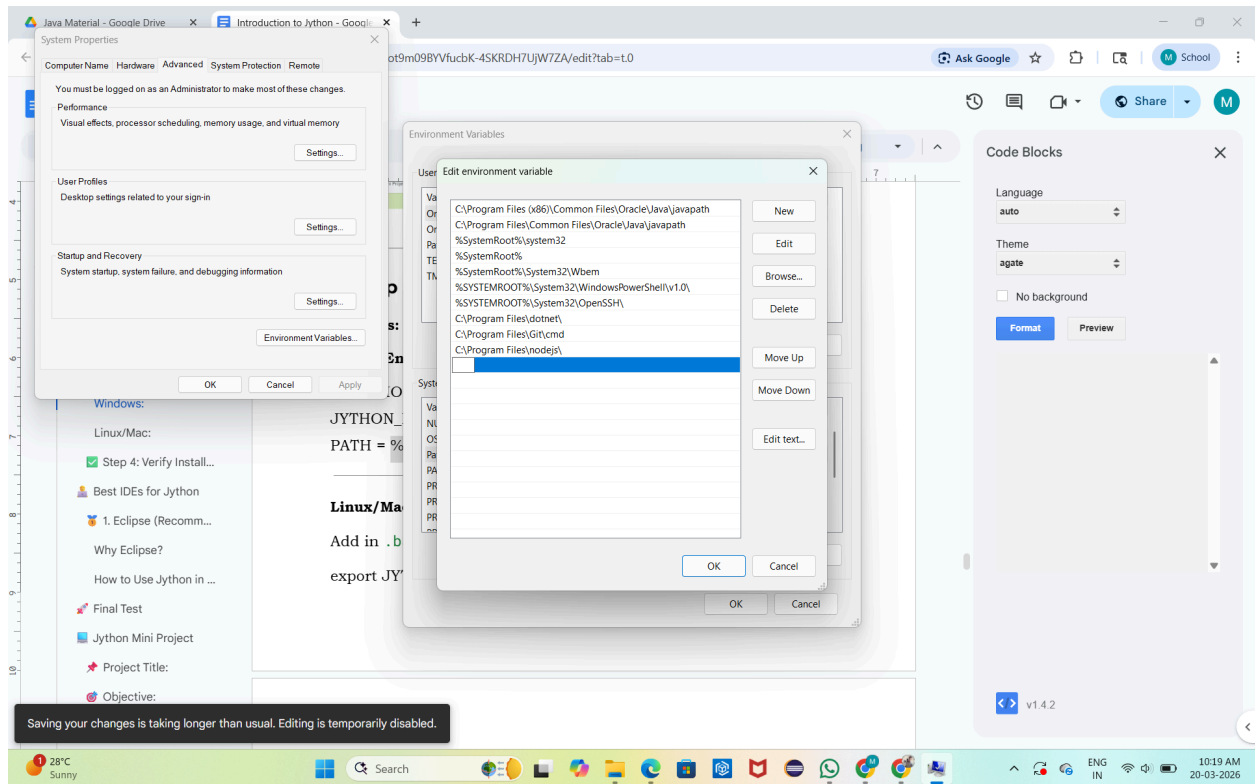
JAVA_HOME = C:\Program Files\Java\jdk

JYTHON_HOME = C:\Jython

PATH = %JYTHON_HOME%\bin







Linux/Mac:

Add in `.bashrc` or `.zshrc`:

```
export JYTHON_HOME=/usr/local/jython  
export PATH=$JYTHON_HOME/bin:$PATH
```

✅ Step 4: Verify Installation

jython

If installed correctly, you'll see:

```
Jython 2.7.x (default...)  
>>>
```



Best IDEs for Jython



1. Eclipse (Recommended for You 👍)

Why Eclipse?

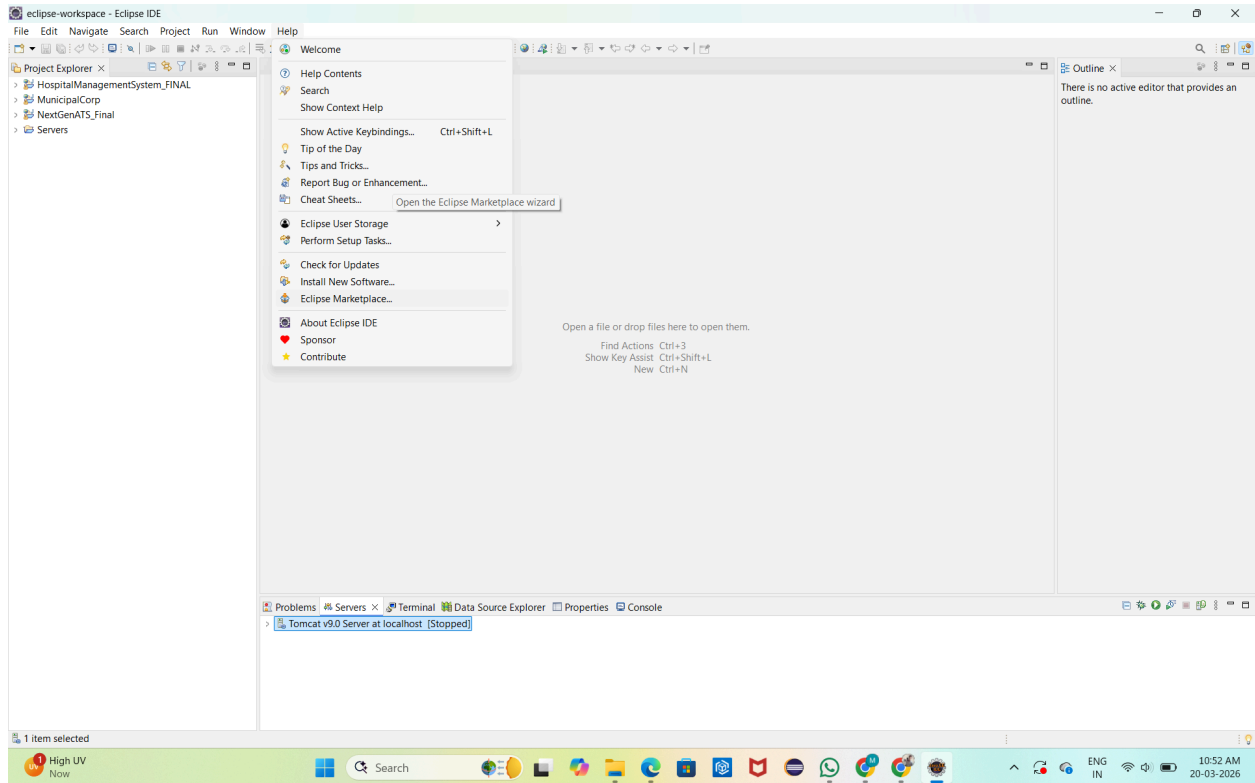
- Best for **Java + Jython integration**
- Supports JVM-based projects
- Easy to configure Jython interpreter

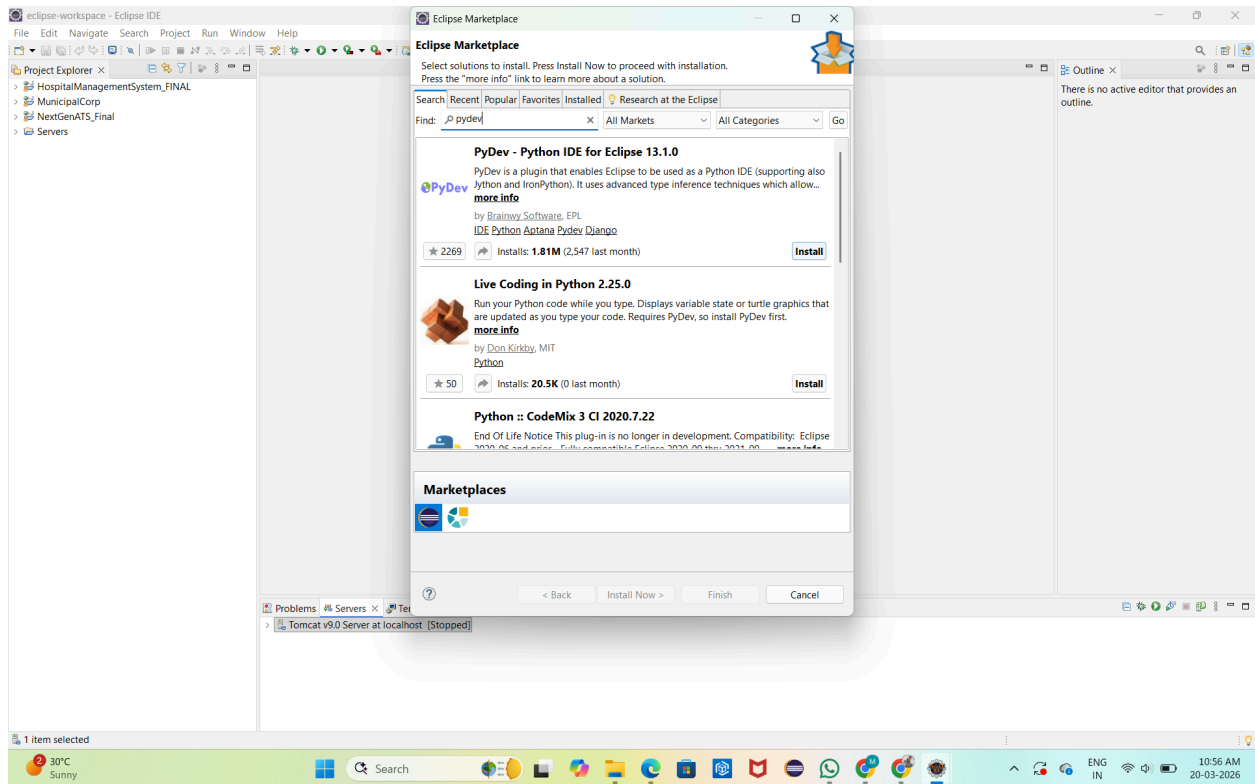
How to Use Jython in Eclipse

1. Install Eclipse IDE (Java version)

2. Go to:

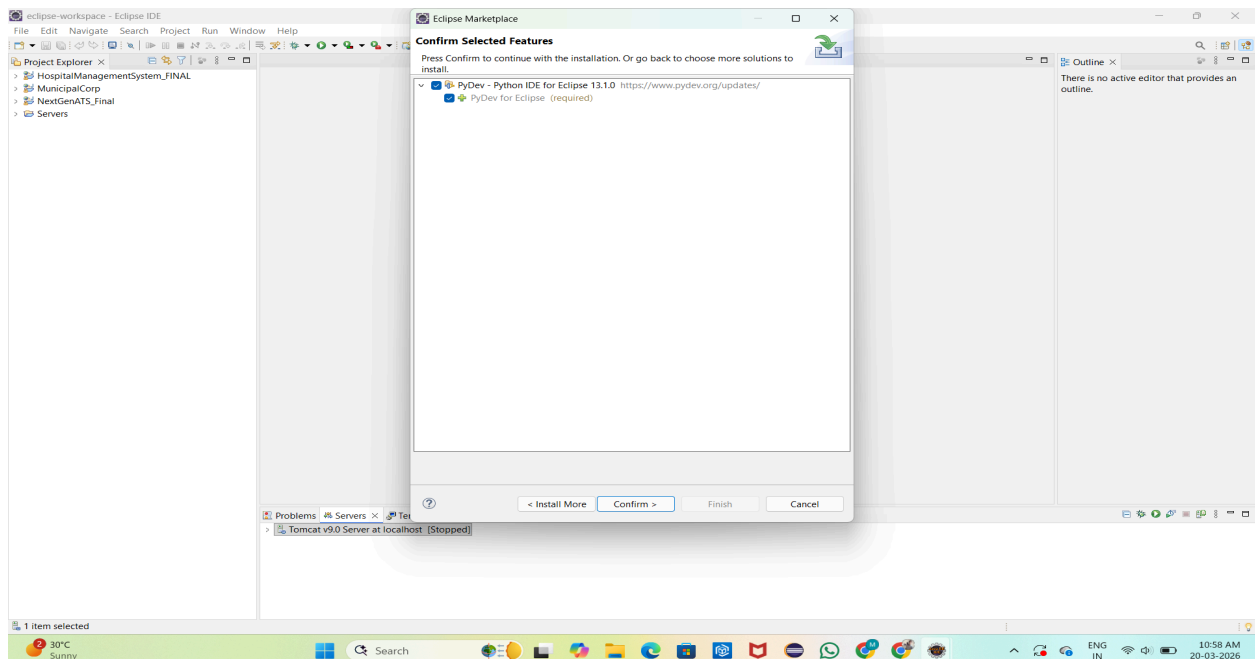
Help → Eclipse Marketplace

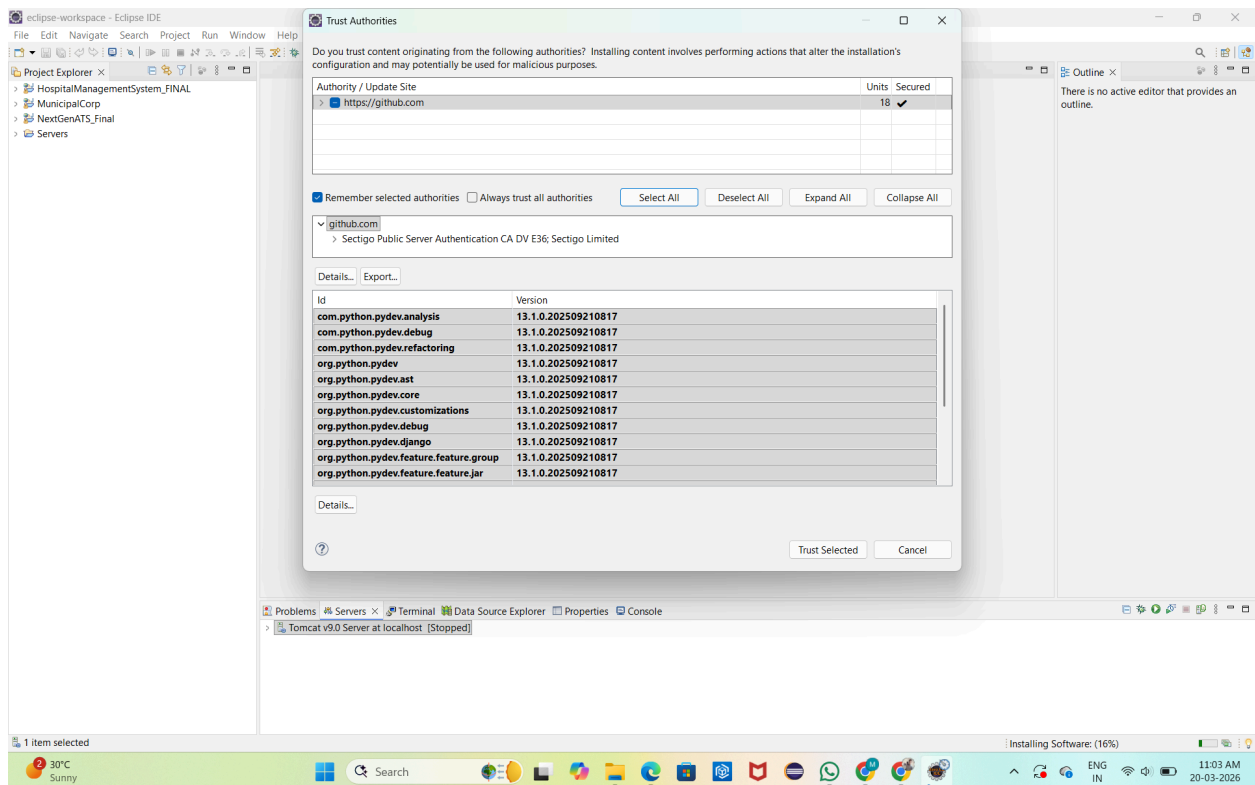
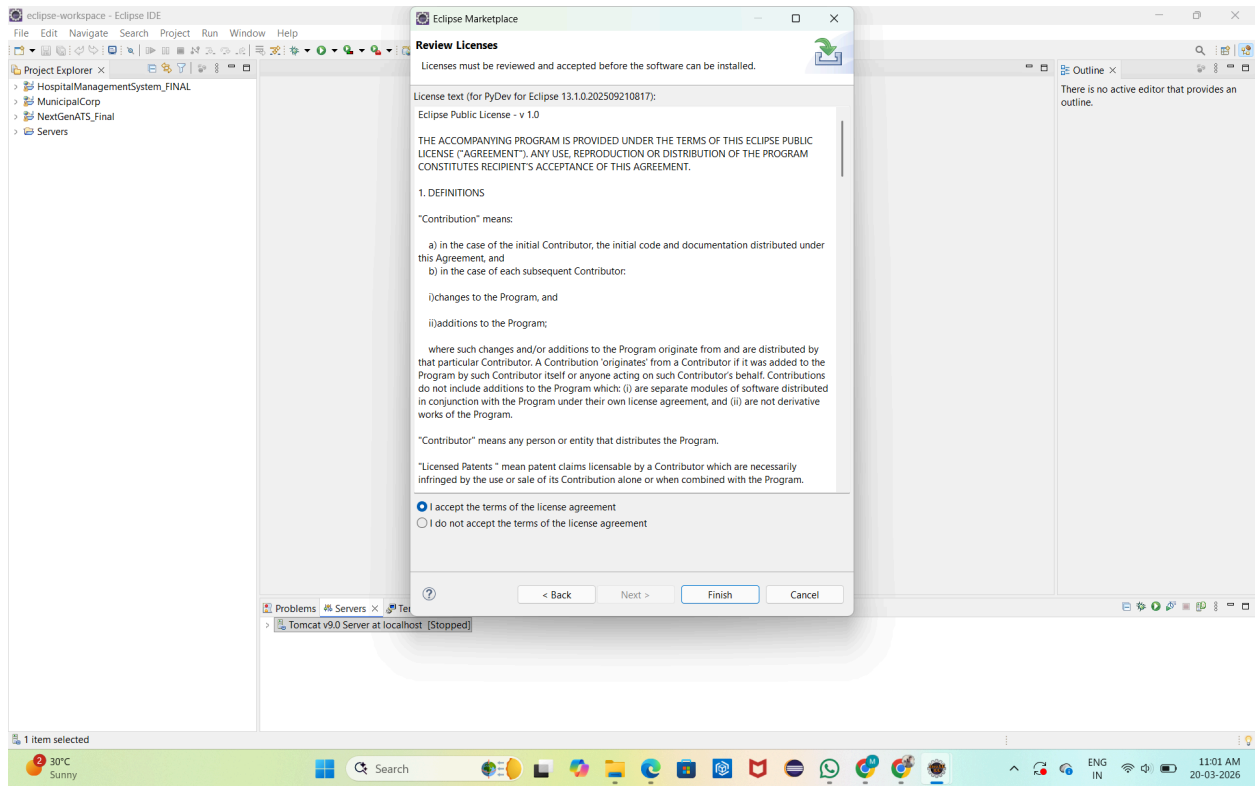


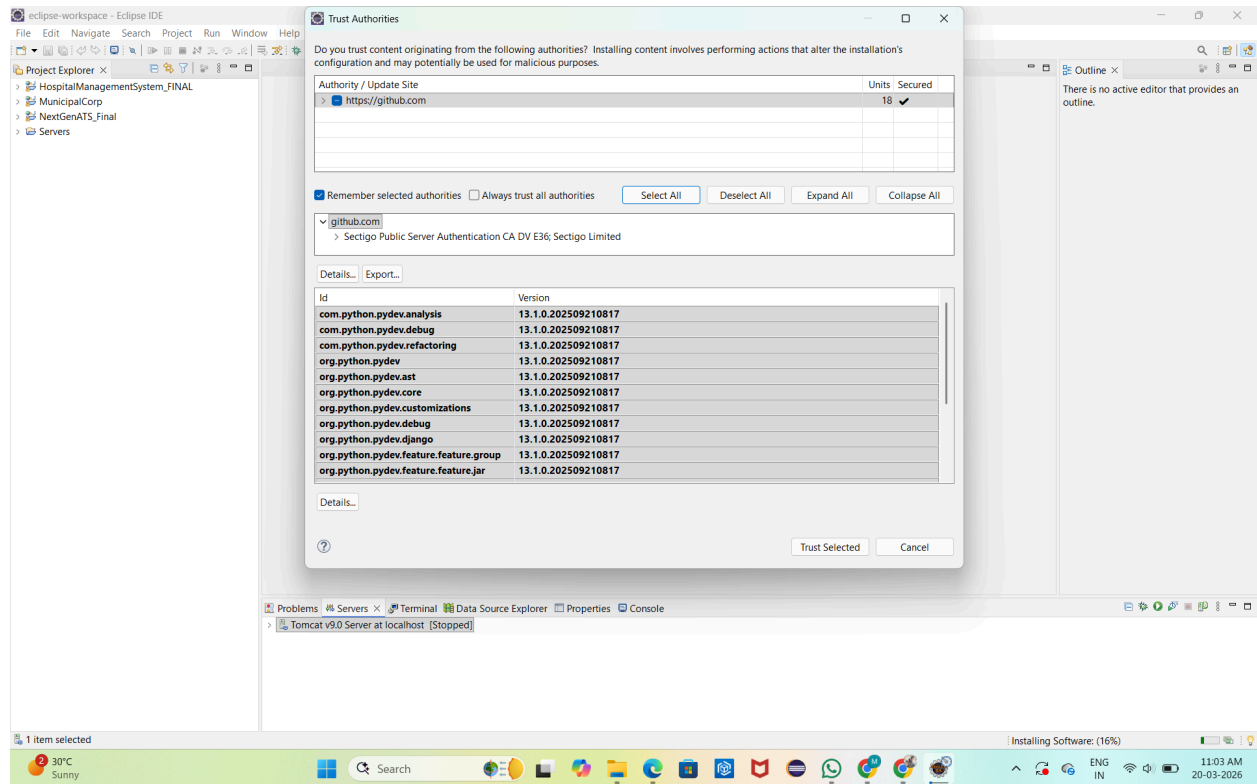


3. Search and install:

👉 **PyDev plugin**

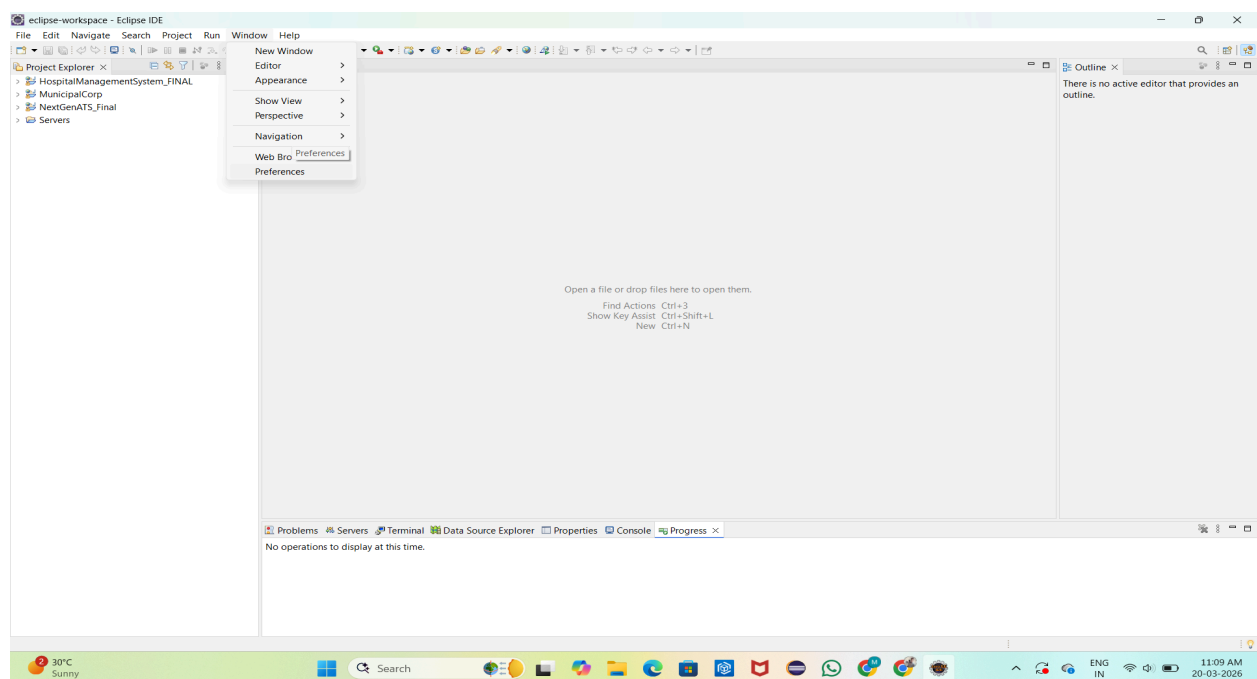


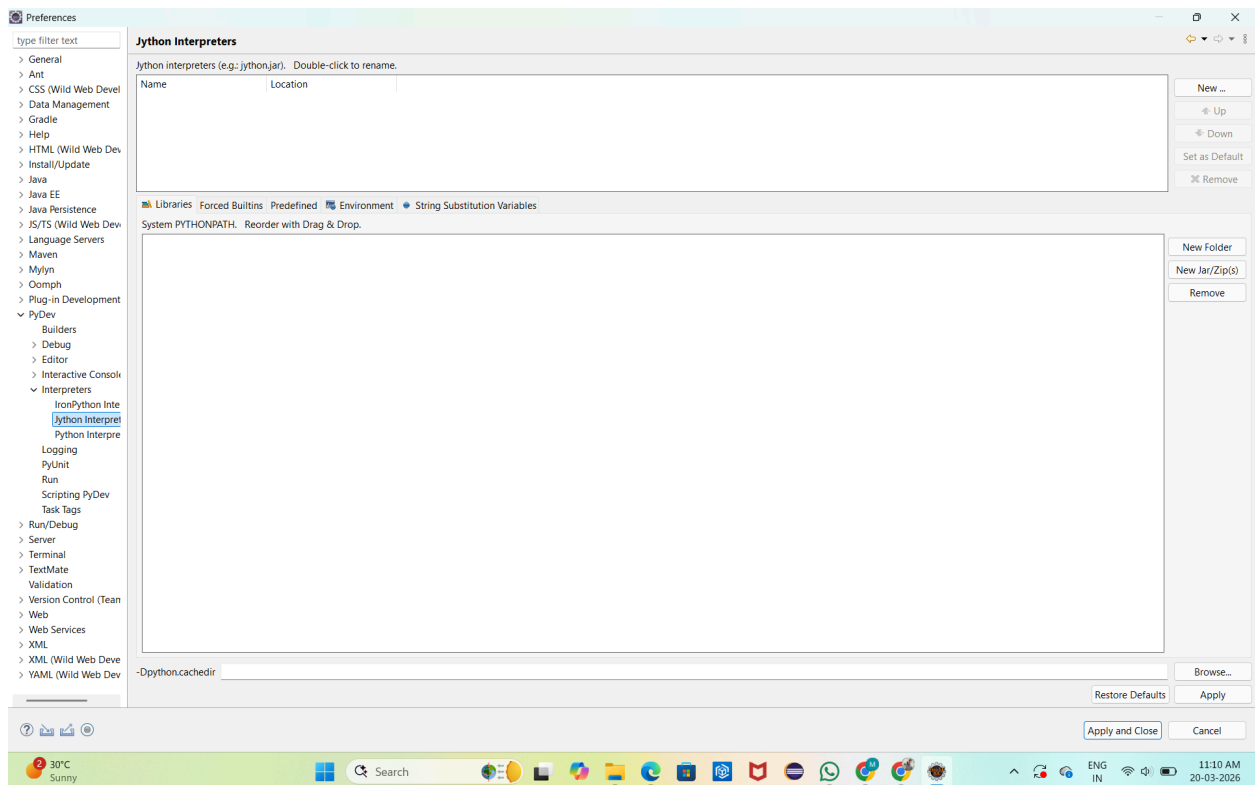
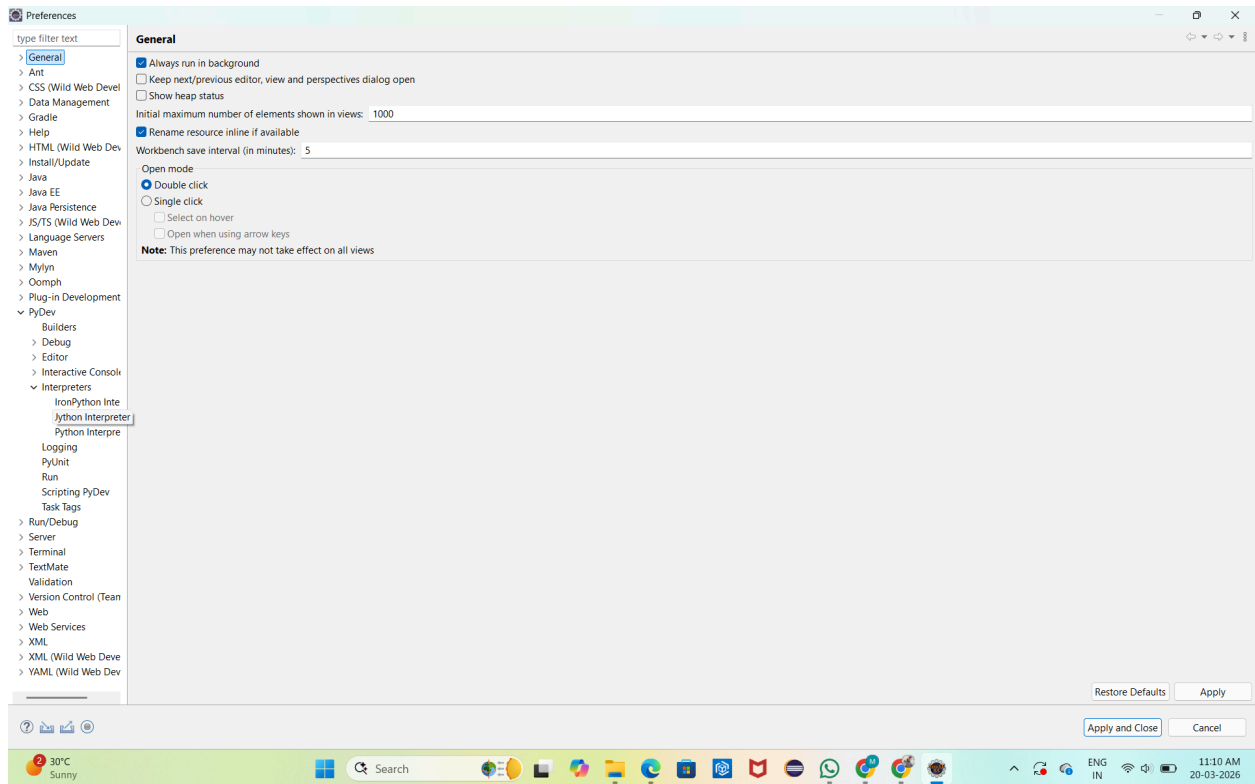




Configure Jython:

Window → Preferences → PyDev → Interpreters → Python

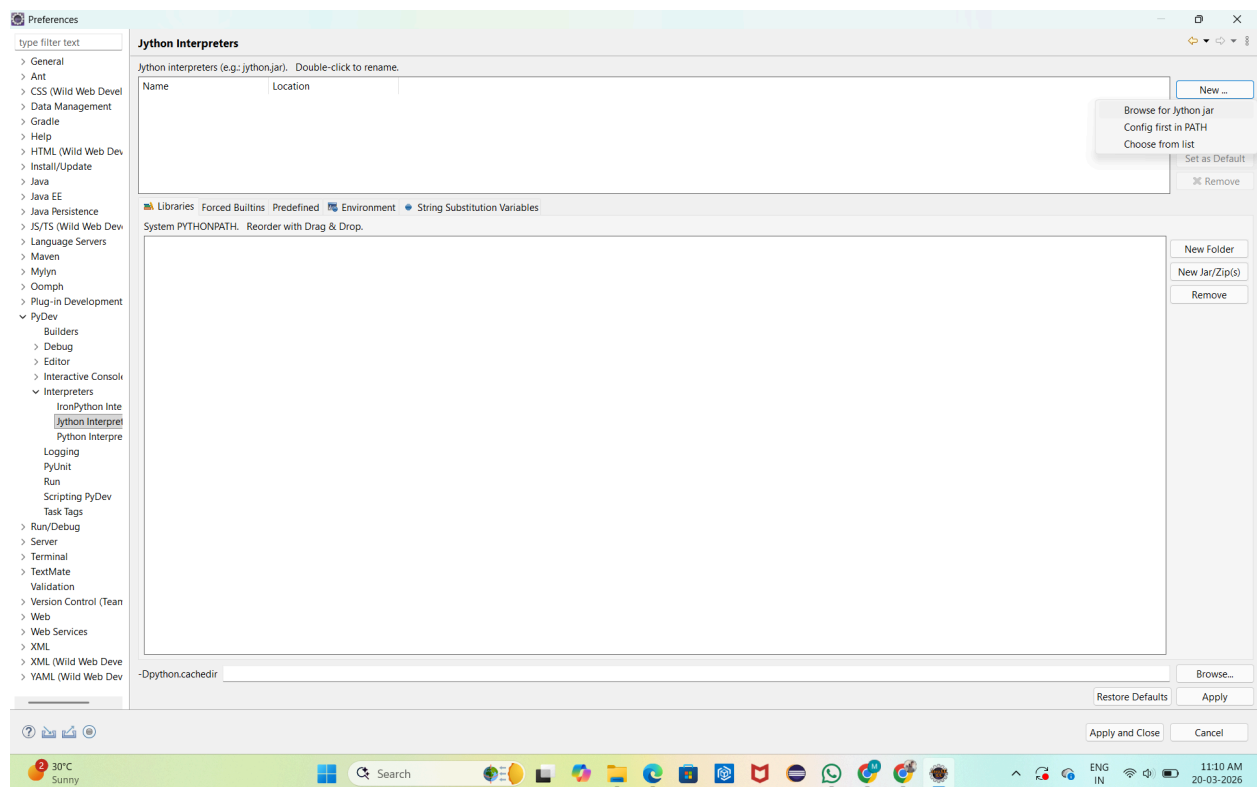


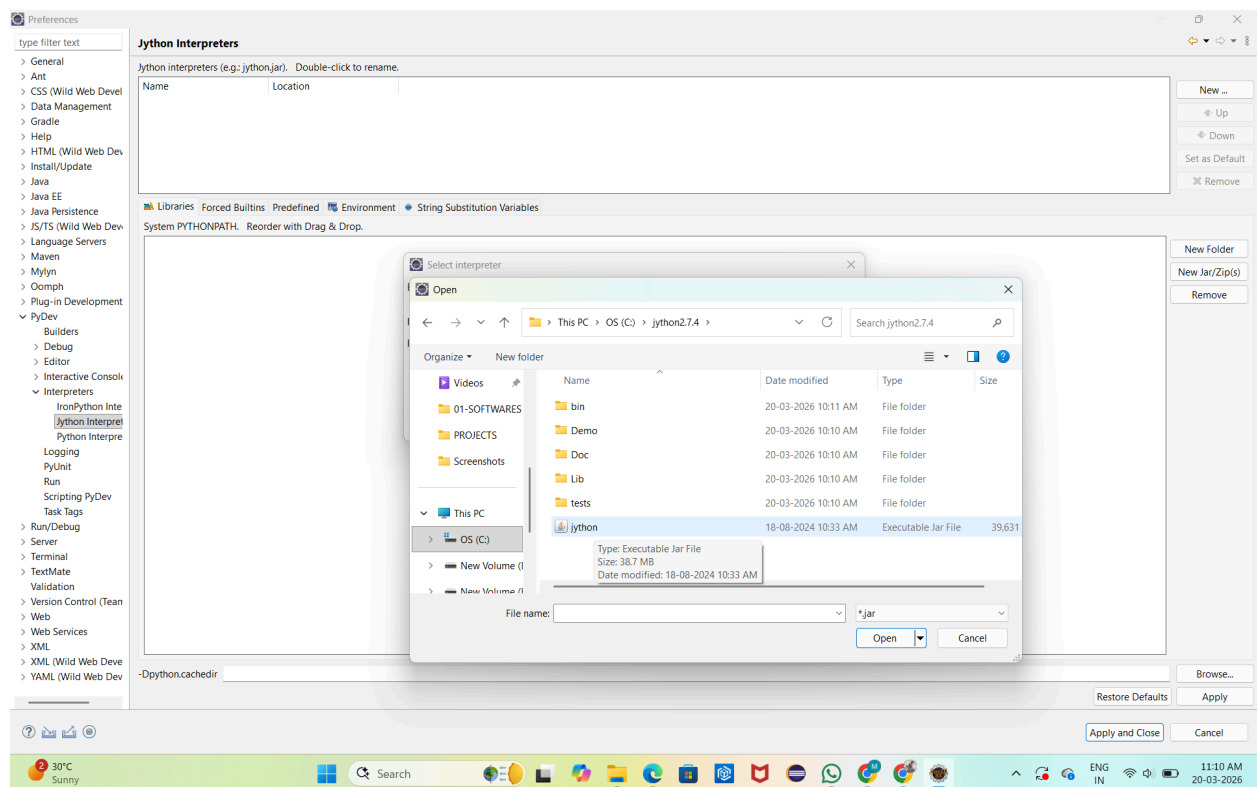
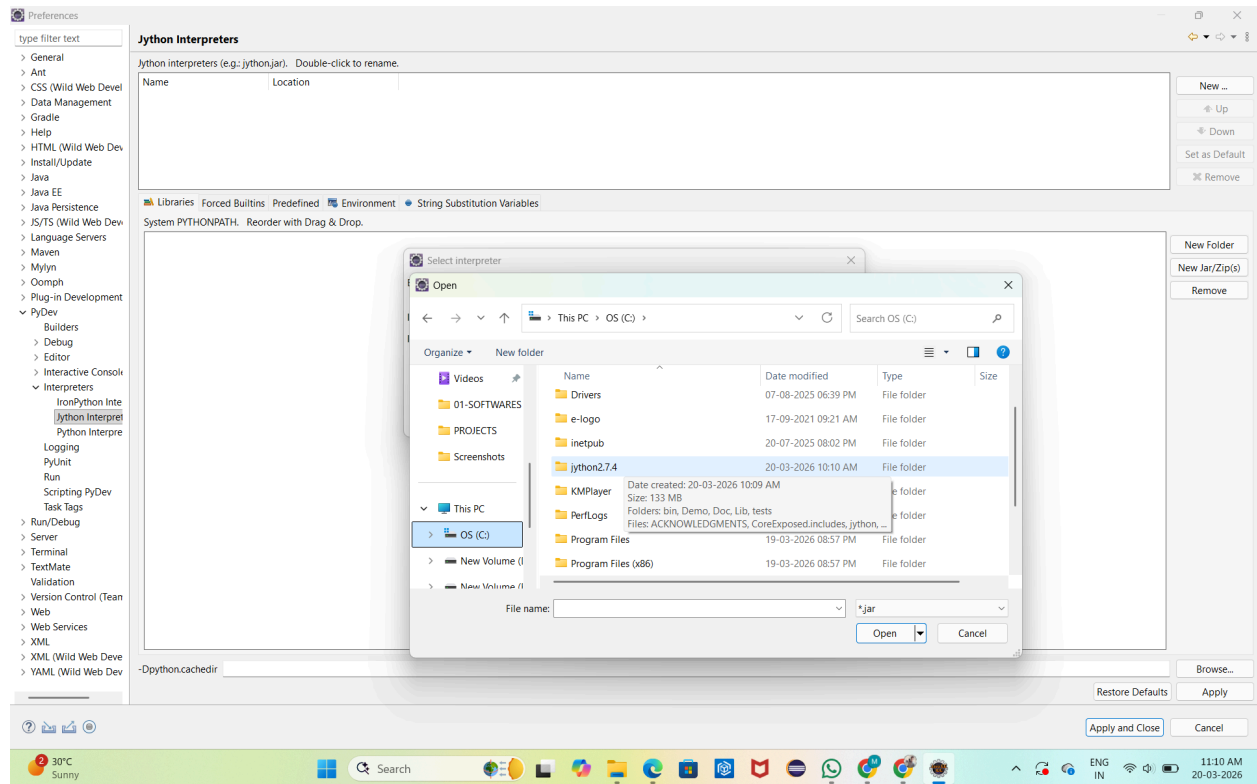


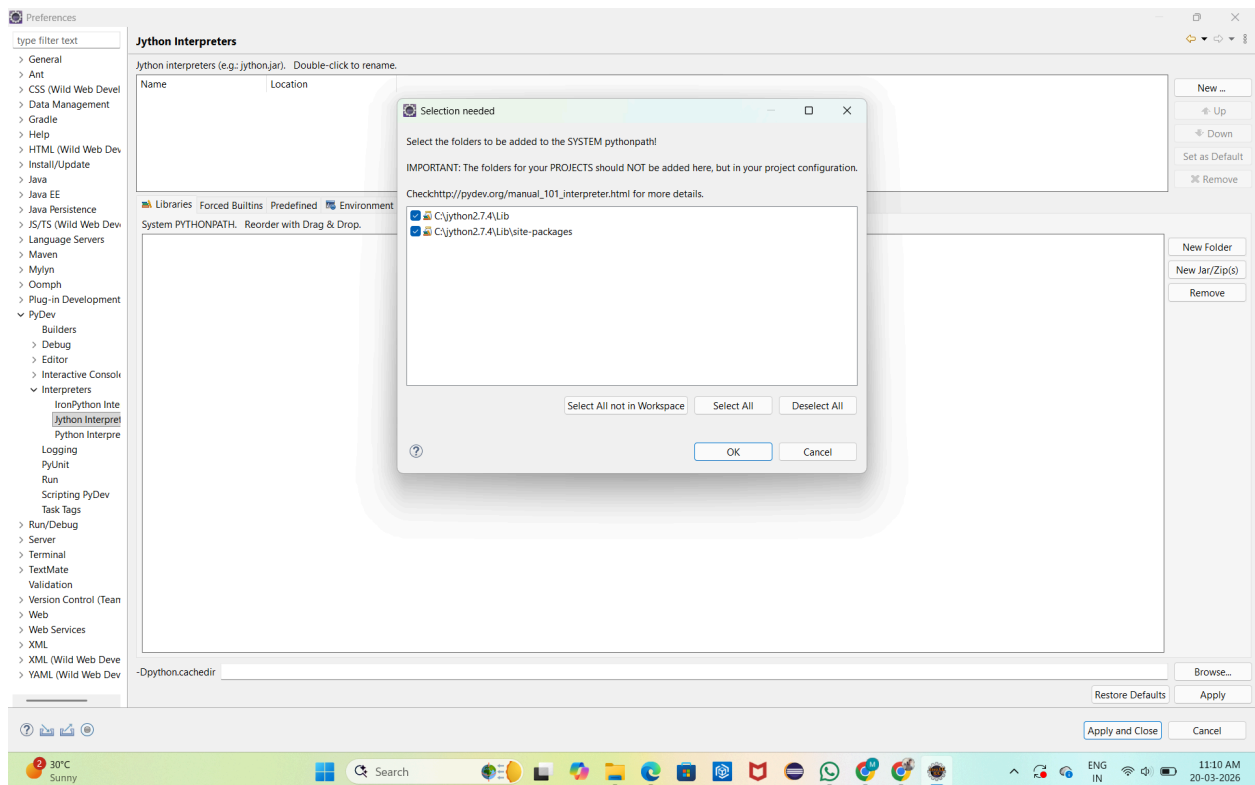
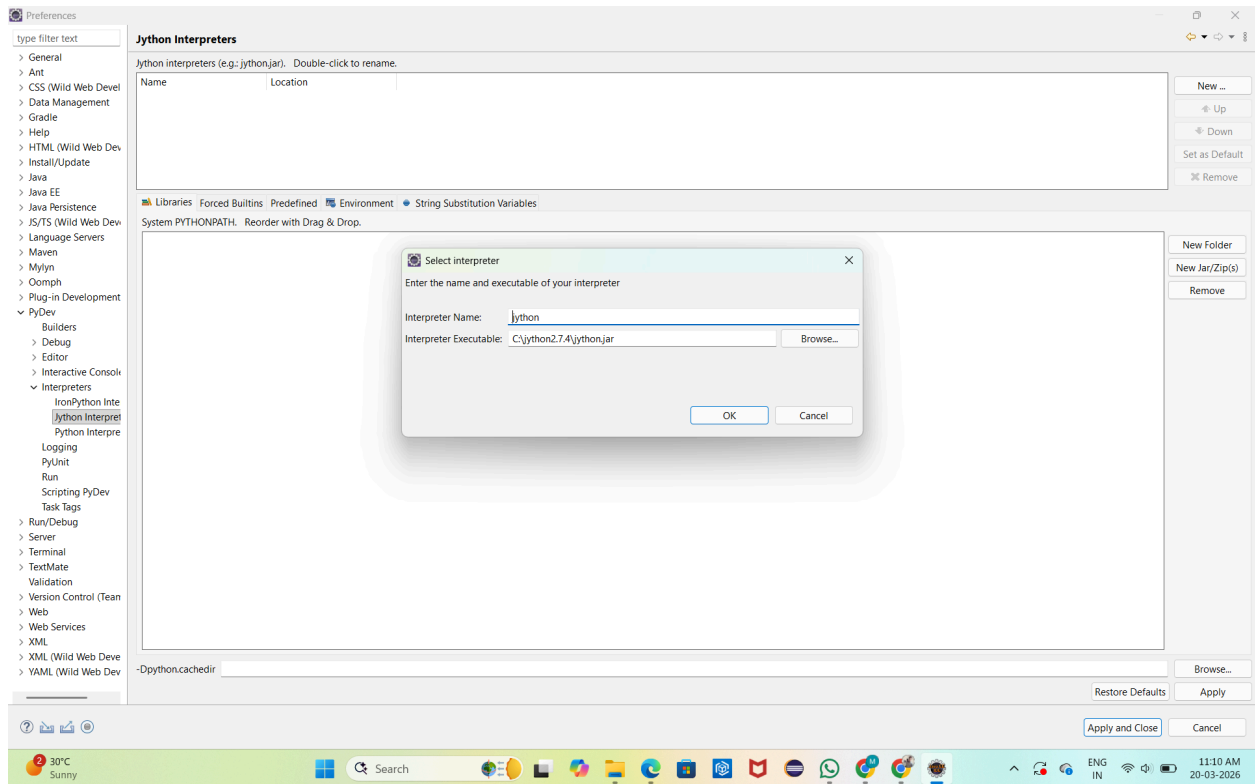
4. Add Jython:

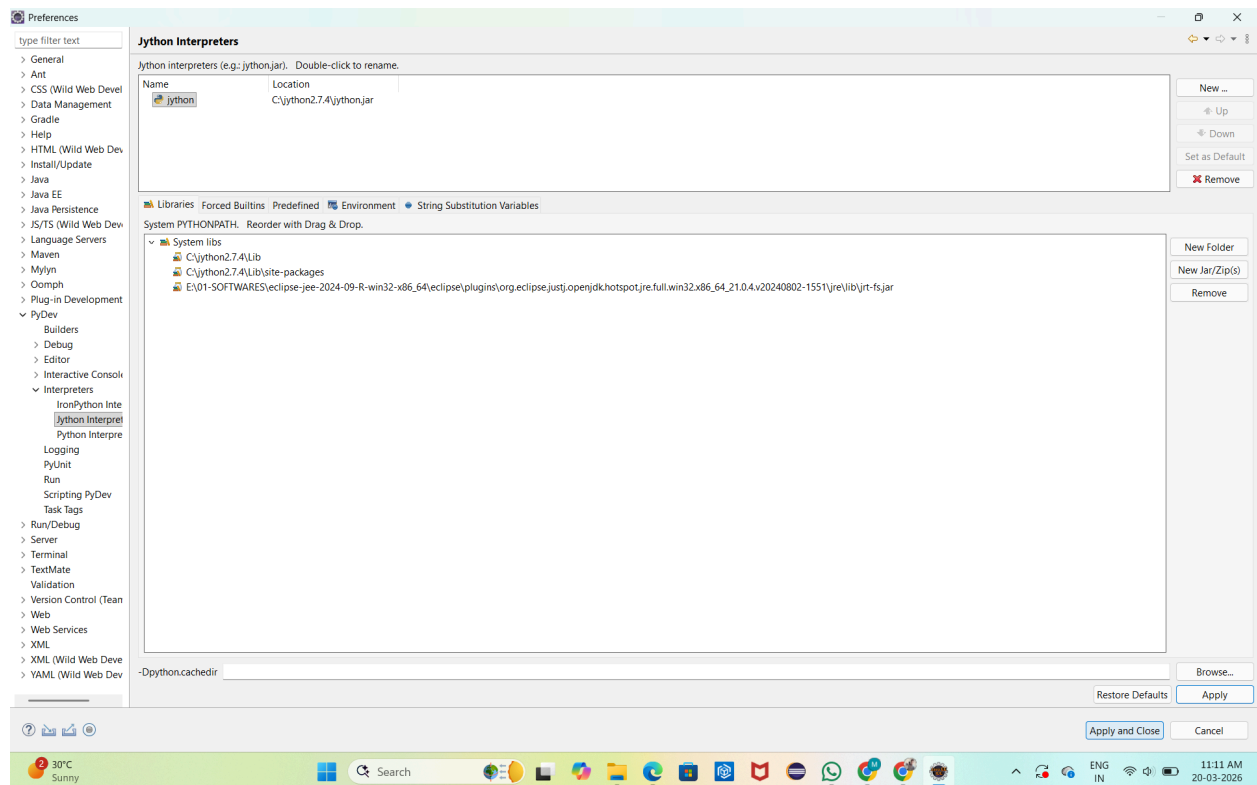
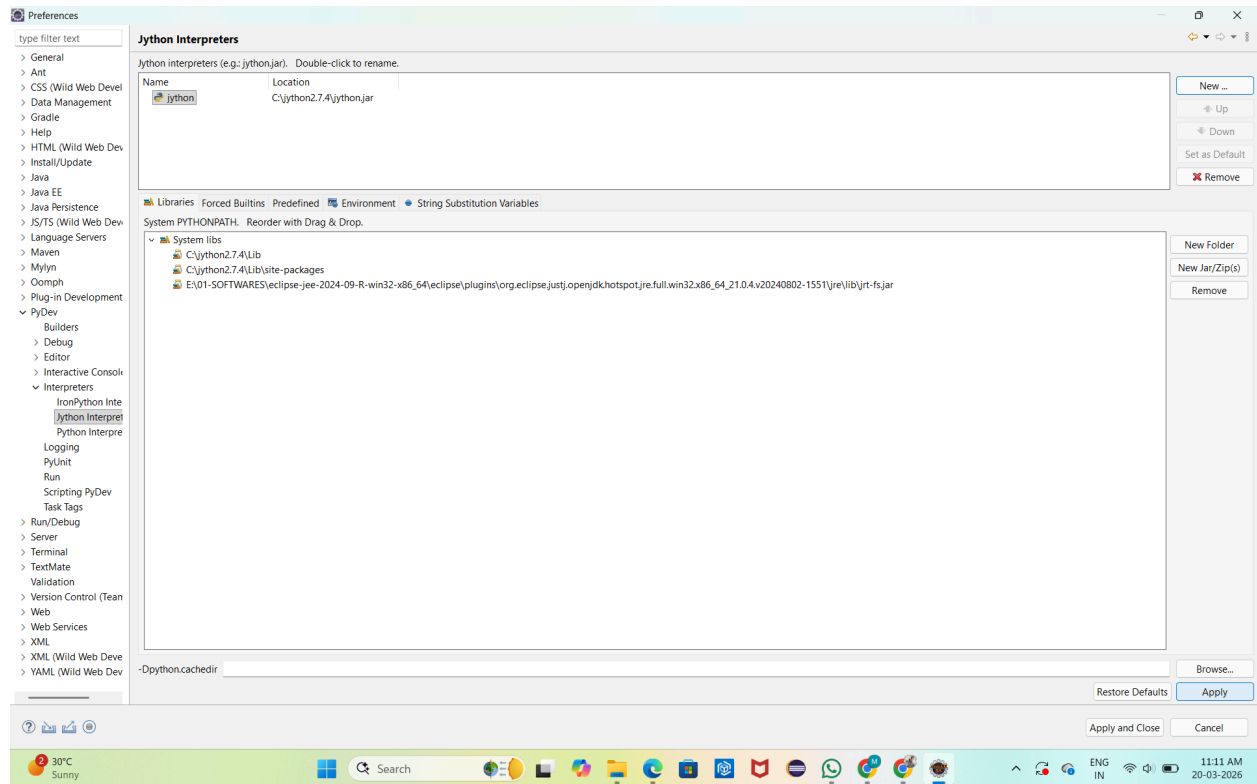
- Select **New**
- Choose Jython executable:

C:\Jython\bin\jython.exe



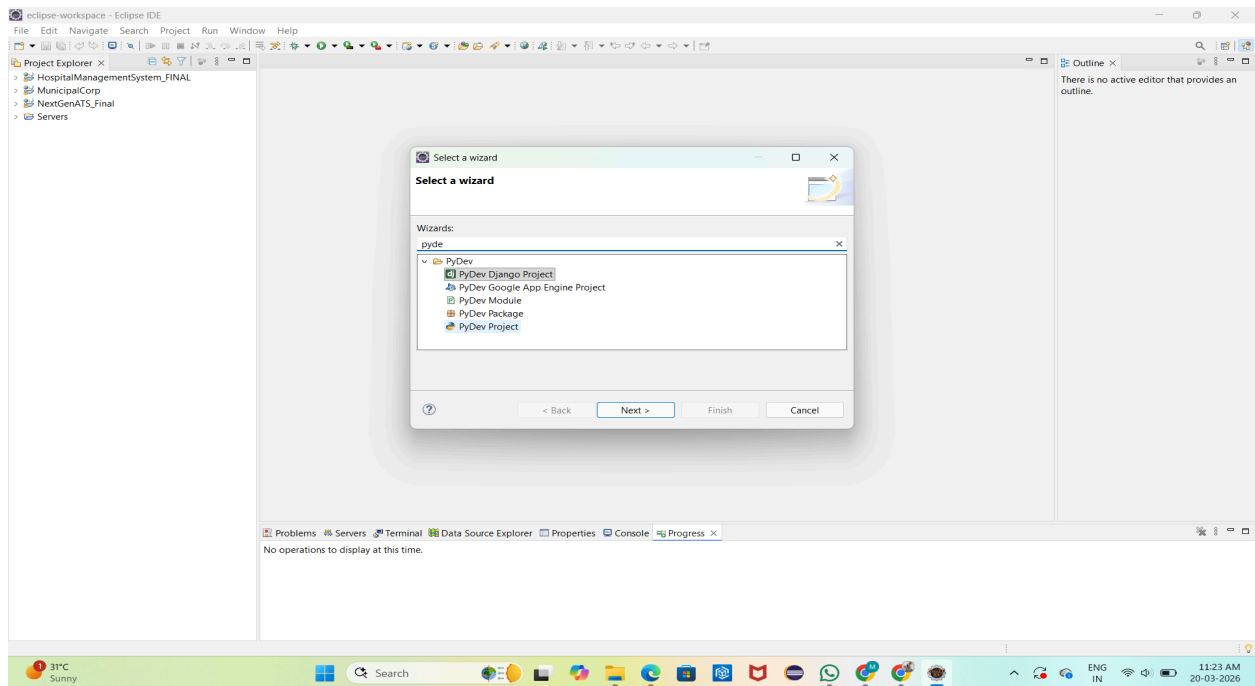
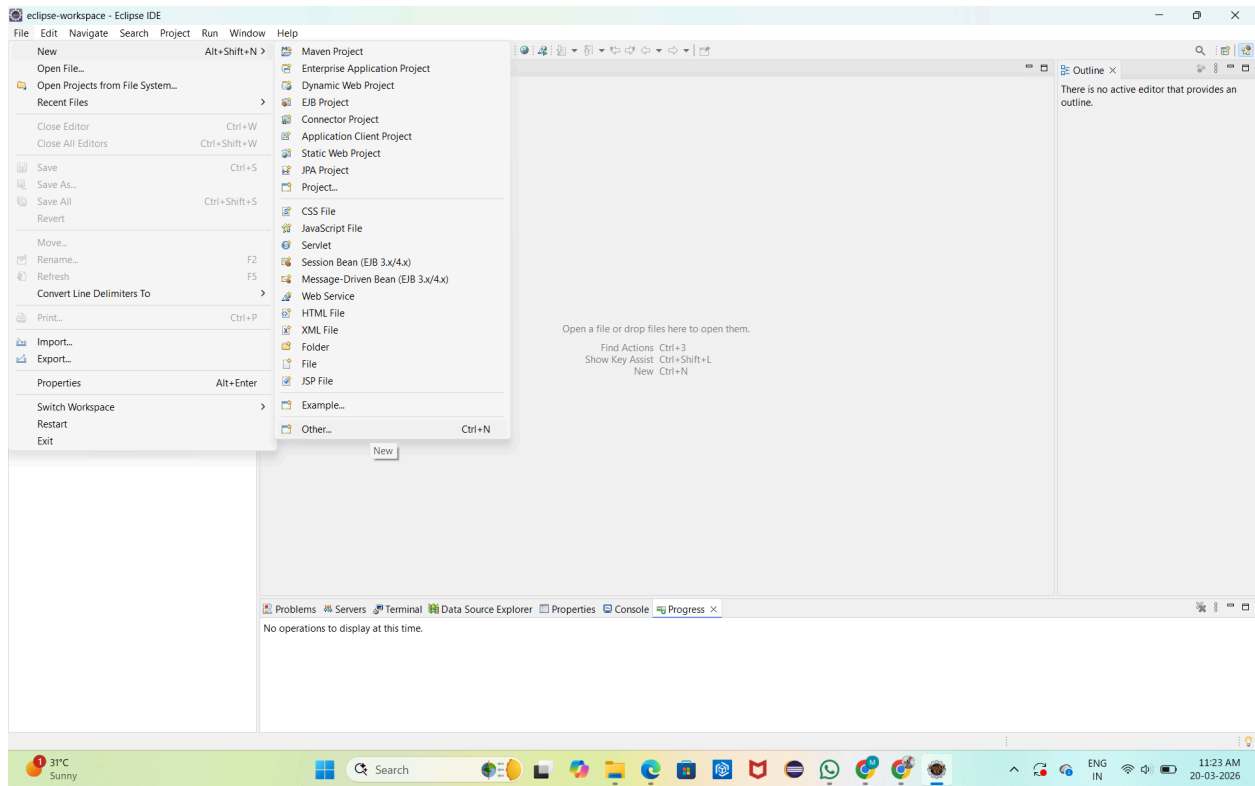


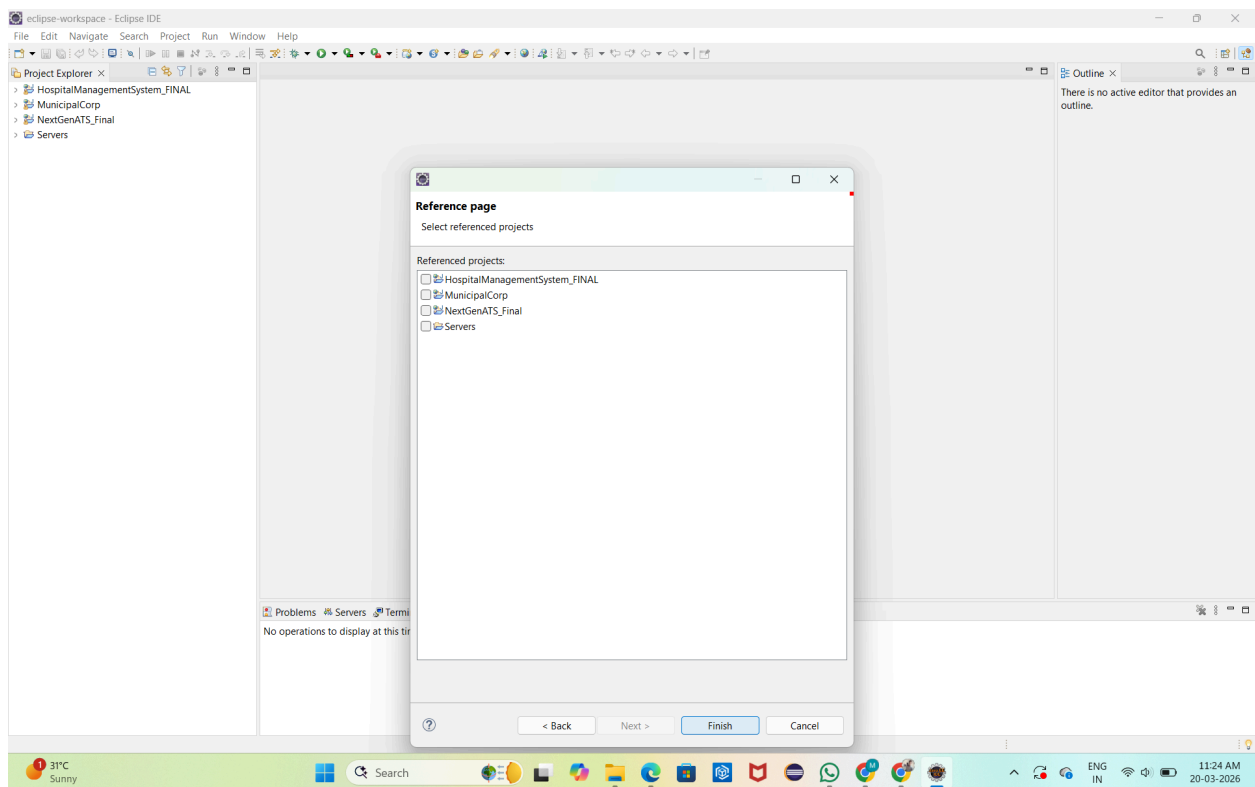
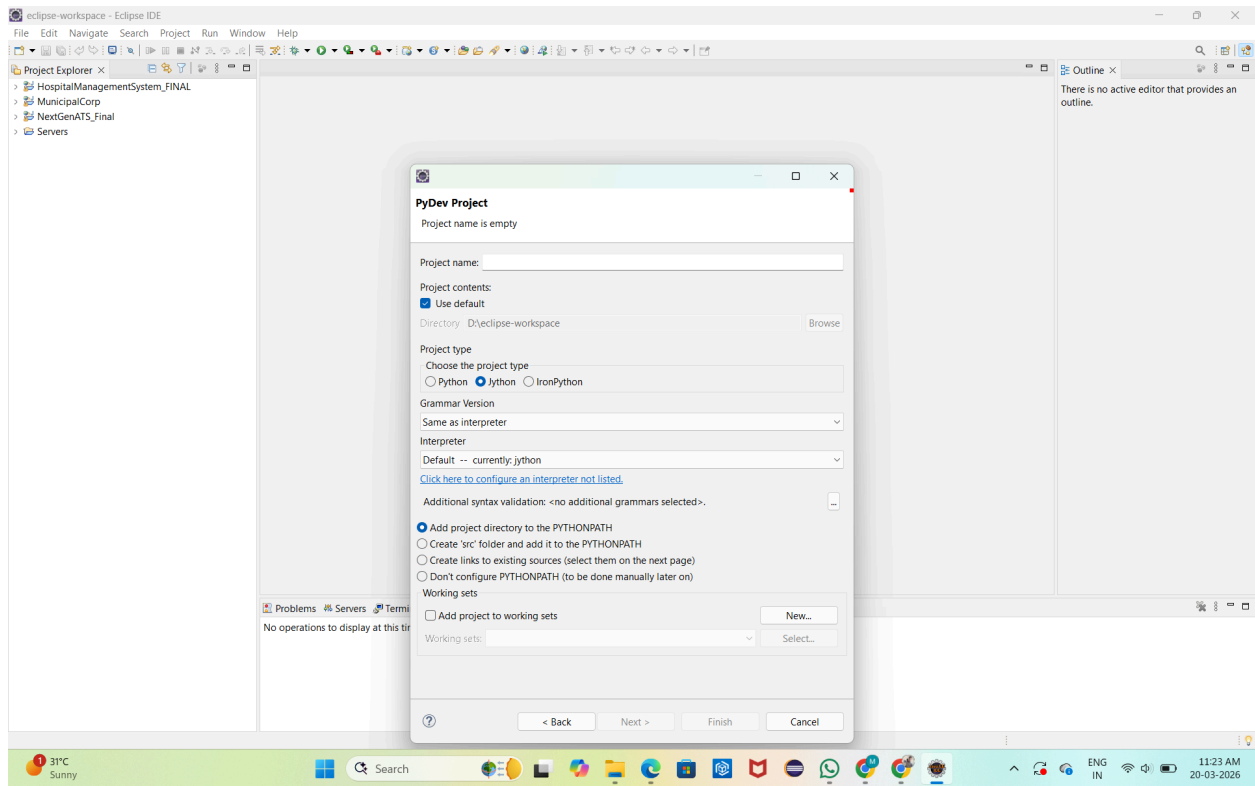


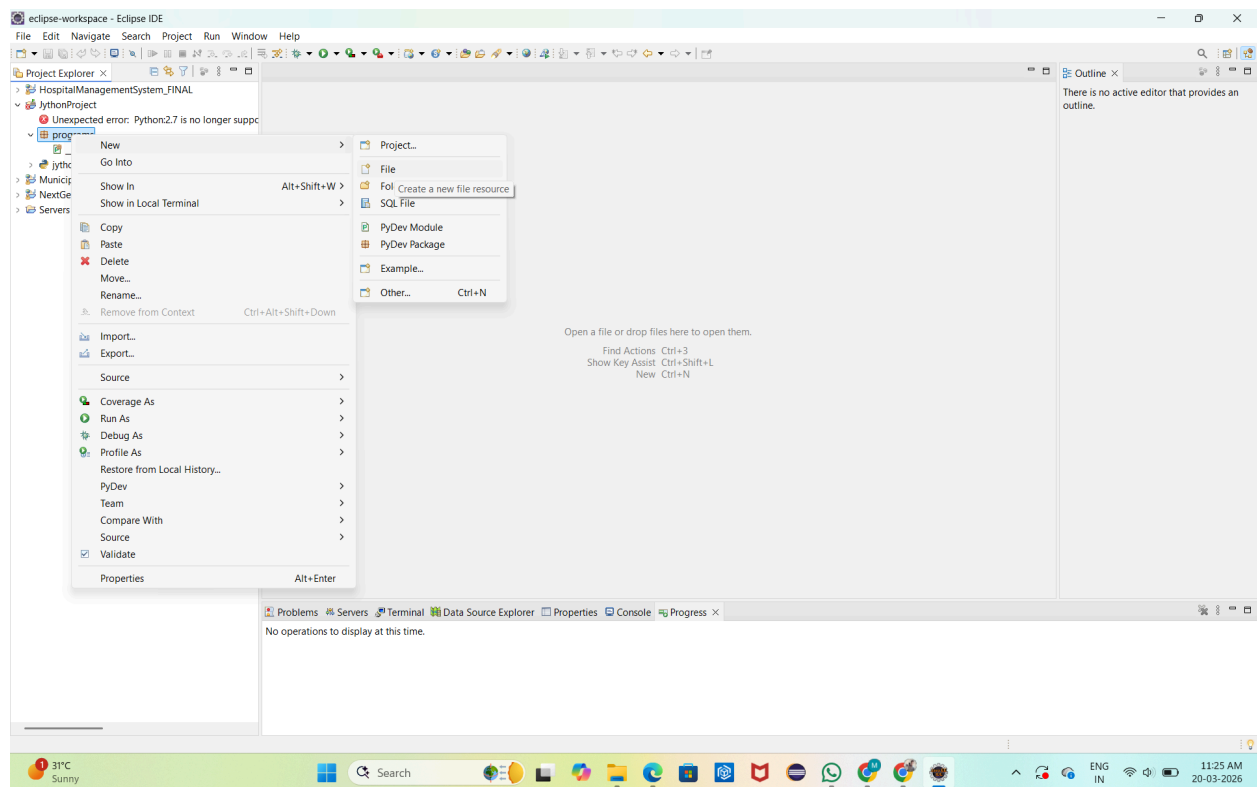
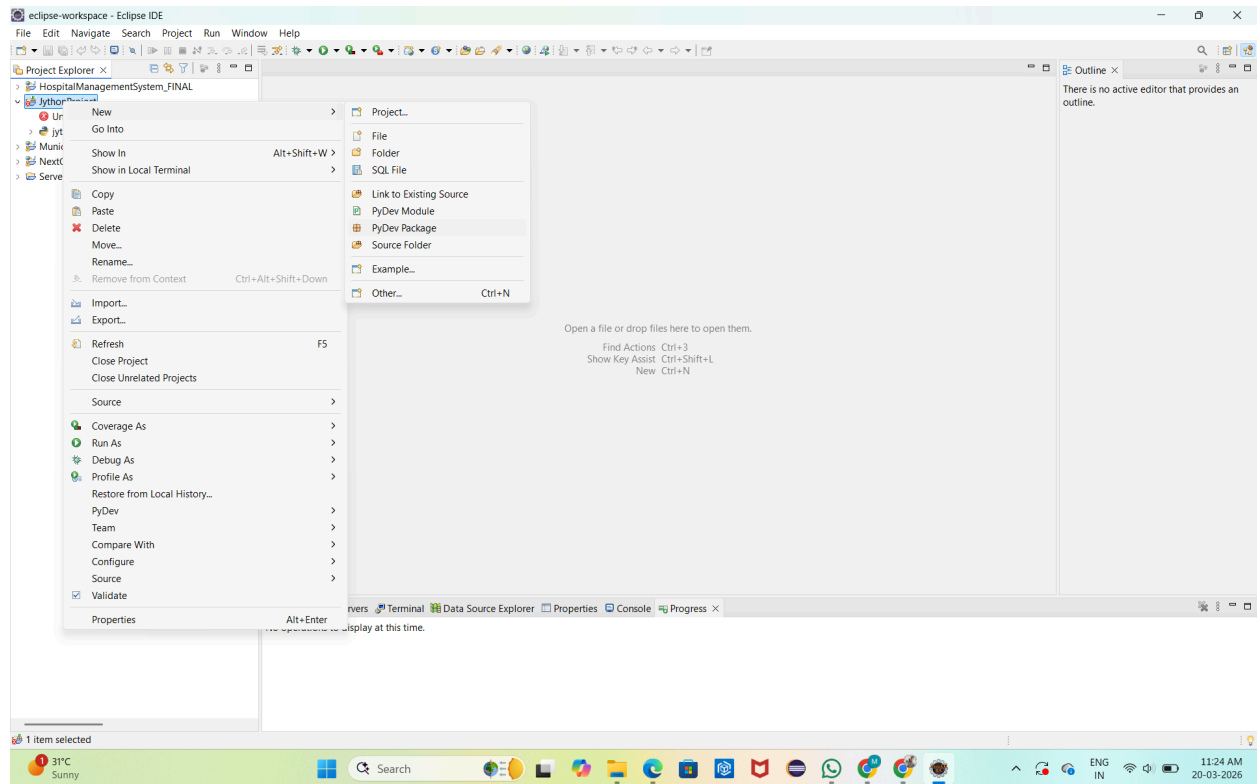


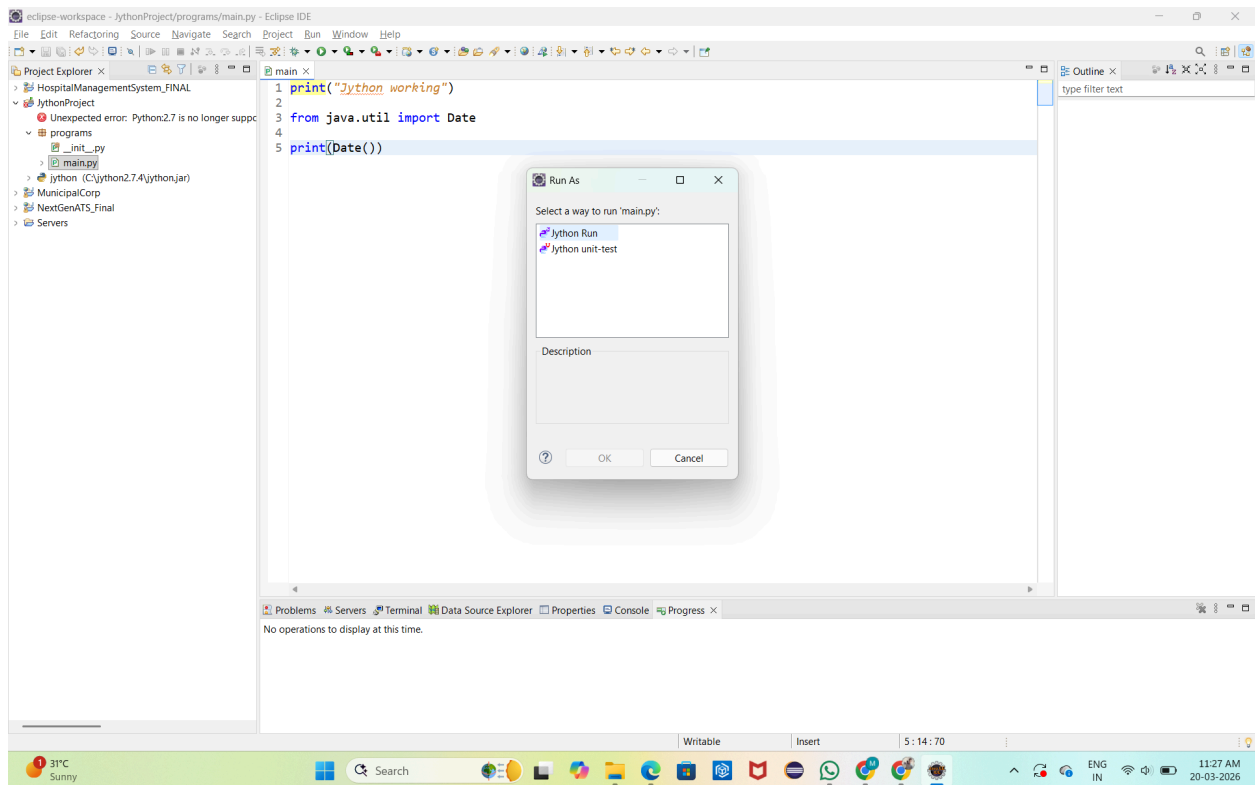
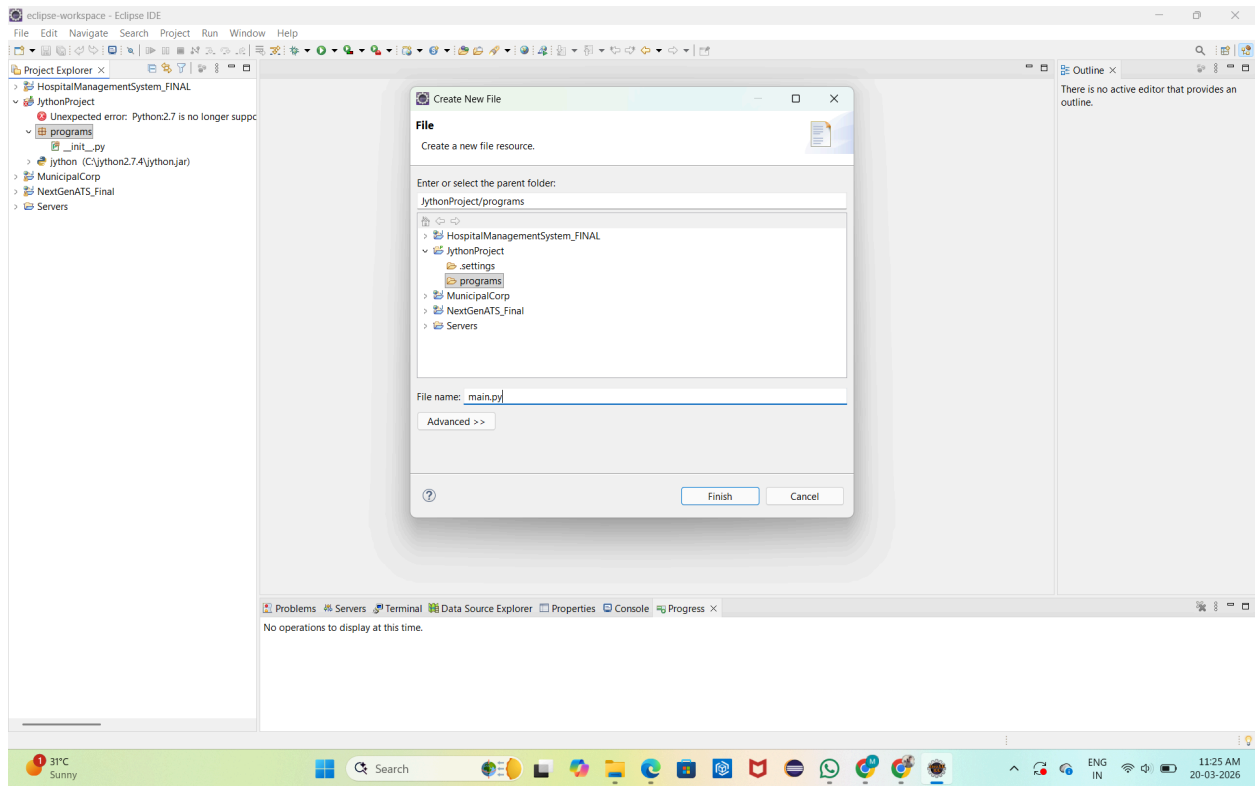
5. Create Project:

File → New → PyDev Project









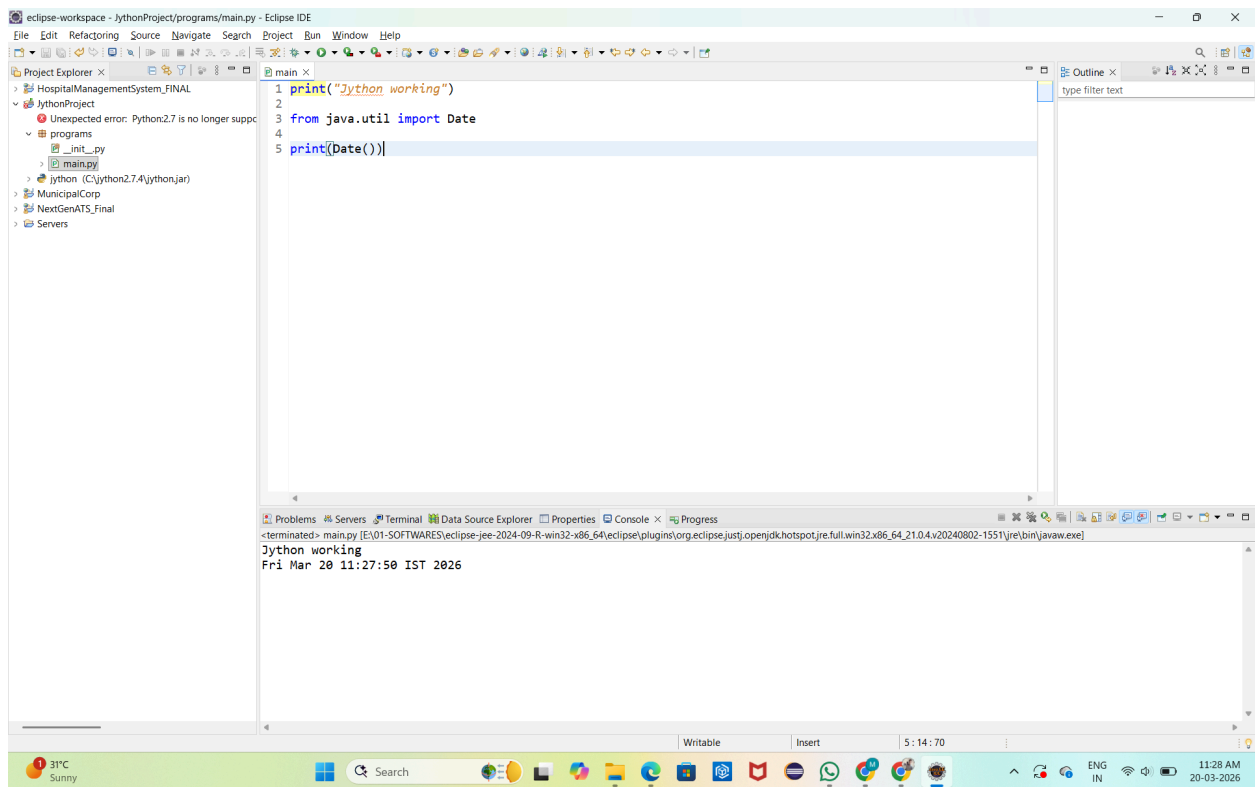
Final Test

Create `main.py`:

```
print("Jython working!")

from java.util import Date
print(Date())
```

Run it 



Jython Mini Project

Project Title:

Student Management System (Jython + Java Integration)

Objective:

- Store student details
 - Use Java classes inside Python
 - Demonstrate real Jython usage
-

Project Structure:

```
JythonProject/  
| — main.py
```

Features:

- Add student
 - View students
 - Uses Java `ArrayList`
-



Code: main.py

```
from java.lang import System
from java.util import ArrayList
from java.util import Scanner

# Create Java ArrayList
students = ArrayList()

scanner = Scanner(System.in)

def add_student():
    print("Enter Student Name:")
    name = scanner.nextLine()

    print("Enter Student Age:")
    age = scanner.nextLine()

    student = "Name: " + name + ", Age: " + age
    students.add(student)

    print("Student Added Successfully!\n")

def view_students():
    if students.isEmpty():
        print("No students found.\n")
    else:
        print("\nStudent List:")
        for s in students:
            print(s)
        print("")

def menu():
    while True:
```

```
print("==== Student Management System ====")
print("1. Add Student")
print("2. View Students")
print("3. Exit")

choice = scanner.nextLine()

if choice == "1":
    add_student()
elif choice == "2":
    view_students()
elif choice == "3":
    print("Exiting...")
    break
else:
    print("Invalid choice!\n")

# Run program
menu()
```

► How to Run

python main.py



Sample Output

```
==== Student Management System ====
1. Add Student
2. View Students
3. Exit
```

Enter choice: 1

Enter Student Name:

Mugilan

Enter Student Age:

22

Student Added Successfully!

Key Concepts You Learned

- Using **Java classes in Python**

```
from java.util import ArrayList
```

- Taking input using Java **Scanner**
- Creating real CLI-based application
- Running Python code on JVM